2.1 Theory of Computation

Automata theory (aka Theory of Computation) is a theoretical branch of Computer Science and Mathematics, which mainly deals with the logic of computation with respect to simple machines, referred to as automata.

Automata* enables the scientists to understand how machines compute the functions and solve problems. The main motivation behind developing Automata Theory was to develop methods to describe and analyze the dynamic behavior of discrete systems. Automata is originated from the word "Automaton" which is closely related to "Automation".

In theoretical computer science, the theory of computation is the branch that deals with whether and how efficiently problems can be solved on a model of computation, using an algorithm. The field is divided into three major branches: automata theory, computability theory and computational complexity theory. In order to perform a rigorous study of computation, computer scientists work with a mathematical abstraction of computers called a model of computation. There are several models in use, but the most commonly examined is the Turing machine.

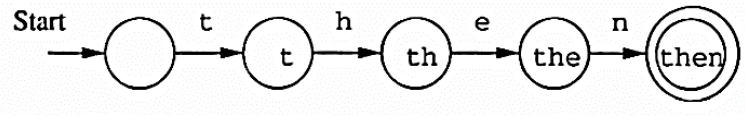
Automata theory

In theoretical computer science, automata theory is the study of abstract machines (or more appropriately, abstract 'mathematical' machines or systems) and the computational problems that can be solved using these machines. These abstract machines are called automata. This automaton consists of:

- states (represented in the figure by circles),
- and transitions (represented by arrows).

As the automaton sees a symbol of input, it makes a transition (or jump) to another state, according to its transition function (which takes the current state and the recent symbol as its inputs).

Uses of Automata: compiler design and parsing.



2.2 Finite Automata

- Finite automata are used to recognize patterns.
- It takes the string of symbol as input and changes its state accordingly. When the desired symbol is found, then the transition occurs.
- At the time of transition, the automata can either move to the next state or stay in the same state.
- Finite automata have two states, Accept state or Reject state. When the input string is processed successfully, and the automata reached its final state, then it will accept.

Formal Definition of FA

A finite automaton is a collection of 5-tuple (Q, ∑, δ, q0, F), where:

- Q: finite set of states
- ∑: finite set of the input symbol
- q0: initial state
- F: final state
- δ: Transition function

Finite Automata Model:

Finite automata can be represented by input tape and finite control.

- **Input tape:** It is a linear tape having some number of cells. Each input symbol is placed in each cell.
- **Finite control:** The finite control decides the next state on receiving particular input from input tape.
- The tape reader reads the cells one by one from left to right, and at a time only one input symbol is read.

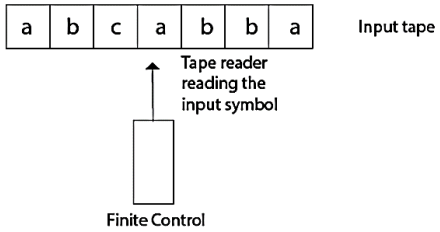


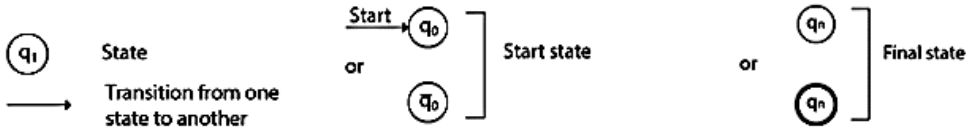
Fig :- Finite automata model

Transition Diagram

A transition diagram or state transition diagram is a directed graph which can be constructed as follows:

- There is a node for each state in Q, which is represented by the circle.
- There is a directed edge from node q to node p labeled a if δ(q, a) = p.
- In the start state, there is an arrow with no source.
- Accepting states or final states are indicating by a double circle.

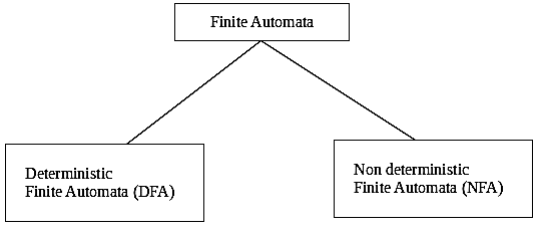
Some Notations that are used in the transition diagram:



Types of Automata:

There are two types of finite automata:

1. DFA (deterministic finite automata)
2. NFA (non-deterministic finite automata)



DFA:

DFA refers to deterministic finite automata. Deterministic refers to the uniqueness of the computation. In the DFA, the machine goes to one state only for a particular input character. DFA does not accept the null move.

NFA:

NFA stands for non-deterministic finite automata. It is used to transmit any number of states for a particular input. It can accept the null move.
Some important points about DFA and NFA:

- Every DFA is NFA, but NFA is not DFA.
- There can be multiple final states in both NFA and DFA.
- DFA is used in Lexical Analysis in Compiler.
- NFA is more of a theoretical concept.

DFA vs NDFA

The following table lists the differences between DFA and NDFA.

DFA	NFA
All transitions are deterministic.	Transitions can be non-deterministic.
Each transition leads to exactly one state.	A transition could lead to a subset of states.
For each state, transitions on all possible symbols(alphabets) should be defined	For each state, not all symbols necessarily have to be defined in the transition function.

Accepts input if the last states in in F	Accepts input if one of the last sates is in F
Sometimes harder to construct because of the numbers of states.	Generally easier thana DFA to construct.
Practical implementation is feasible.	Practical implementation has to be deterministic (so needs conversion to DFA)
Back tracking is allowed in DFA	Back tracking is not allowed in NFA
Requires more memory	Less memory

2.3 DFA

In DFA, for each input symbol, one can determine the state to which the machine will move. Hence, it is called Deterministic Automaton. As it has a finite number of states, the machine is called Deterministic Finite Machine or Deterministic Finite Automaton.

Formal Definition of a DFA

A DFA can be represented by a 5-tuple (Q, ∑, δ, q0, F) where −

- Q is a finite set of states.
- ∑ is a finite set of symbols called the alphabet.
- δ is the transition function where δ: Q × ∑ → Q
- q0 is the initial state from where any input is processed (q0 ∈ Q).
- F is a set of final state/states of Q (F ⊆ Q).

Graphical Representation of a DFA

- A DFA is represented by digraphs called state diagram.
- The vertices represent the states.
- The arcs labeled with an input alphabet show the transitions.
- The initial state is denoted by an empty single incoming arc.
- The final state is indicated by double circles.

Example

Let a deterministic finite automaton be →

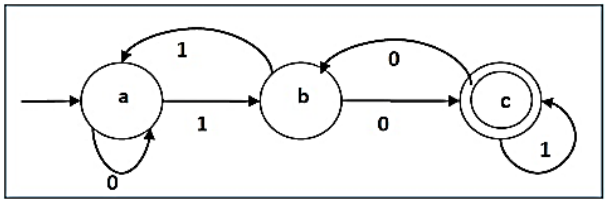
Q = {a, b, c},
∑ = {0, 1},
q_0 = {a},
F = {c}, and

Transition function δ as shown by the following table −

Q	0	1
a	a	b
b	c	a
c	b	c

Or the transition function can be written as:
δ(a, 0) → a
δ(a, 1) → b
δ(b, 0) → c
δ(b, 1) → a
δ(c, 0) → b

Its graphical representation would be as follows –

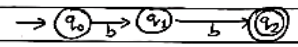


2.4 DFA Numerical Problems:

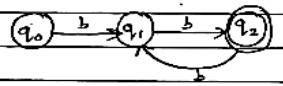
1. **Design a DFA that accepts the language given by: L= {w ∈ {a, b}* : w has even numbers of b}**

Sol^n: Here, L = { abb, bb, bbbb, bab, bba, bbabb, baabab, ---- }

First let us design for bb



Again design for bbbb



Hence the DFA is

M = { Q, Σ, δ, q0, F }

where,

Q = { q0, q1, q2 }

Σ = { a, b }

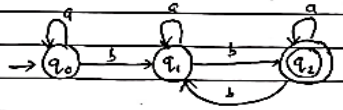
q0 = { q0 }

F = { q2 }

δ :

Q	a	b
q0	q0	q1
q1	q1	q2
q2	q2	q1

State diagram is :



2. **Design a DFA that accepts the language given by: L= {w ∈ {1, 0}* : w ends with 100 }**

Here,

L = { 100, 1100, 0100, 10100, 010100, ---- }

Let M = { Q, Σ, δ, q0, F } where

Q = { q0, q1, q2, q3 }

Σ = { 0, 1 }

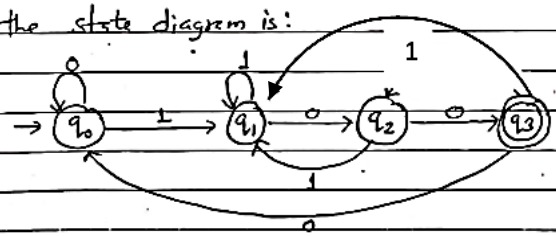
q0 = { q0 }

F = { q3 }

δ :

Q	0	1
q0	q0	q1
q1	q2	q1
q2	q3	q1
q3	q0	q1

Hence, the state diagram is:



Verification:

10100 → Verified ends at q3, accepted

10101 → ends at q1, rejected

3. **Construct a DFA starting with aba, ∑ = a, b.**

Here,

L = { aba, abaa, abab, ---- }

Let M = { Q, Σ, δ, q0, F }

where,

Q = { q0, q1, q2, q3, qØ }

Σ = { a, b }

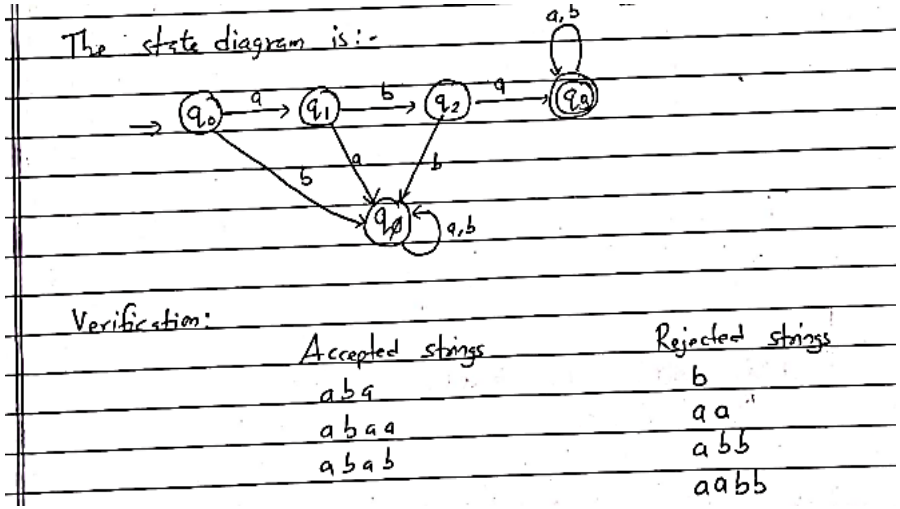
q0 = { q0 }

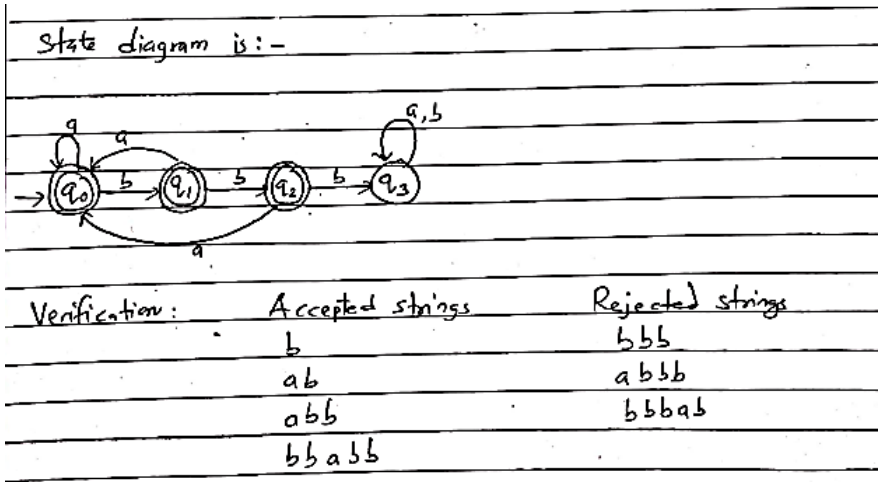
F = { q3 }

and

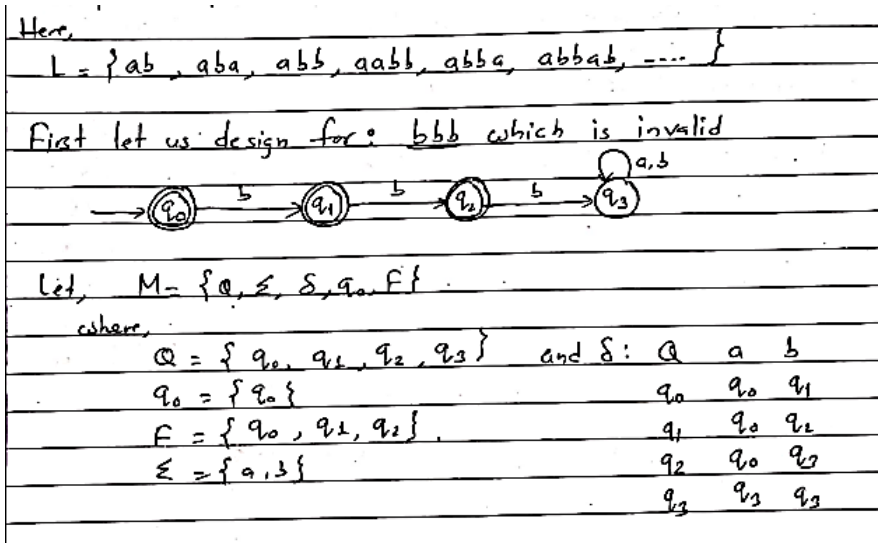
δ :

Q	a	b
q0	q1	qØ
q1	qØ	q2
q2	q3	qØ
q3	q3	q3
qØ	qØ	qØ

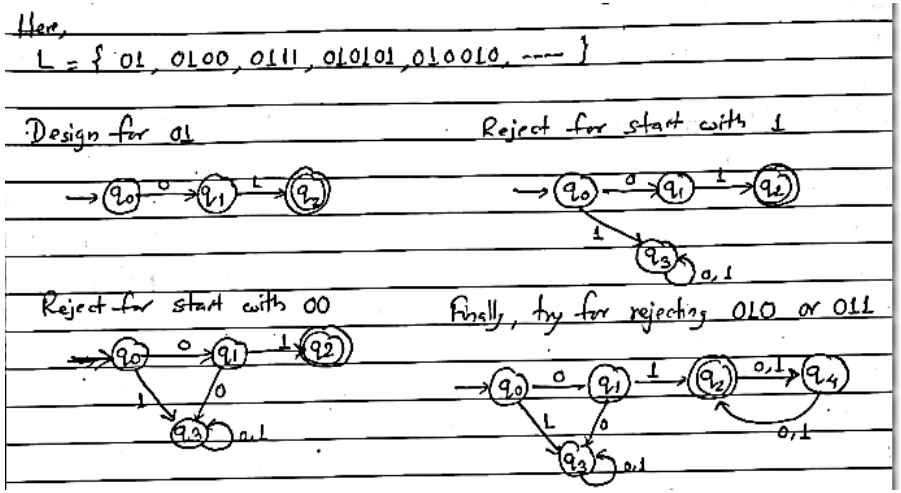




4. Design a DFA that accepts the language given by: L= {w ∈ {a, b}* : w does not contain three consecutive b }



5. Design a DFA that accepts the language given by: L= {w ∈ {0, 1}* : w starts with 01 having even length}



Hence, DFA is:

$M = \{Q, \Sigma, \delta, q_0, F\}$

where,

$Q = \{q_0, q_1, q_2, q_3, q_4\}$

$\Sigma = \{0, 1\}$

$q_0 = \{q_0\}$

$F = \{q_2\}$

and

δ:

Q	0	1
q_0	q_1	q_3
q_1	q_3	q_2
q_2	q_4	q_4
q_3	q_3	q_3
q_4	q_2	q_2

Verification:

$0111 \Rightarrow \delta(q_0, 0111)$
$\vdash (q_1, 111)$
$\vdash (q_2, 11)$
$\vdash (q_4, 1)$
$\vdash (q_2, \epsilon)$

Since q_2 lies in final state
DFA accepts

$010 \Rightarrow \delta(q_0, 010)$
$\vdash (q_1, 10)$
$\vdash (q_2, 0)$
$\vdash (q_4, \epsilon)$

Since q_4 is not a final state, DFA rejects

6. Design a DFA that accepts the language given by: L= {w ∈ {0, 1}* : w starts with 01 }

Here,

$L = \{01, 0100, 010, 0111, 010111 \ldots\}$

The DFA is:

$M = \{Q, \Sigma, \delta, q_0, F\}$

where,

$Q = \{$

$\Sigma = \{0, 1\}$

$q_0 = q_0$

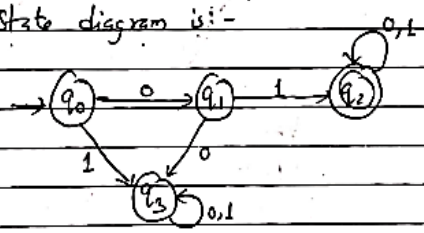
$F = \{q_2\}$

and

δ:

Q	0	1
q_0	q_1	q_3
q_1	q_3	q_2
q_2	q_2	q_2
q_3	q_3	q_3

State diagram is:-



Verification:

Accepted string	Rejected string
0101	001
$\delta(q_0, 0101)$	$\delta(q_0, 001)$
$\vdash (q_1, 101)$	$\vdash (q_1, 01)$
$\vdash (q_3, 01)$	$\vdash (q_3, 1)$
$\vdash (q_2, 1)$	$\vdash (q_3, \epsilon)$
$\vdash (q_2, \epsilon)$	
Since, q_2 is final state, DFA accepts	Since, q_2 is not final state, DFA rejects

Tutorial:

Page no: 29 : Example no: 2.1 – 2.11

2.5 NDFA

n NDFA, for a particular input symbol, the machine can move to any combination of the states in the machine. In other words, the exact state to which the machine moves cannot be determined. Hence, it is called Non-deterministic Automaton. As it has finite number of states, the machine is called Non-deterministic Finite Machine or Non-deterministic Finite Automaton.

Formal Definition of an NDFA

An NDFA can be represented by a 5-tuple (Q, ∑, δ, q_0, F) where −

- **Q** is a finite set of states.
- **∑** is a finite set of symbols called the alphabets.
- **δ** is the transition function where δ: Q × ∑ → 2^Q
  (Here the power set of Q (2^Q) has been taken because in case of NDFA, from a state, transition can occur to any combination of Q states)
- **q_0** is the initial state from where any input is processed (q_0 ∈ Q).
- **F** is a set of final state/states of Q (F ⊆ Q).

Graphical Representation of an NDFA: (same as DFA)

- An NDFA is represented by digraphs called state diagram.
- The vertices represent the states.
- The arcs labeled with an input alphabet show the transitions.
- The initial state is denoted by an empty single incoming arc.
- The final state is indicated by double circles.

Example

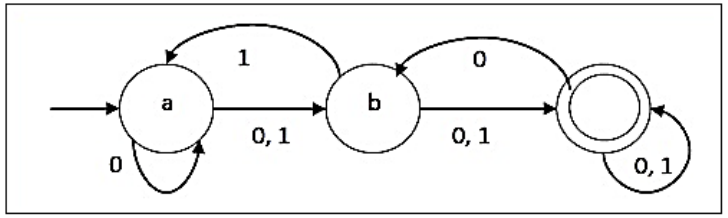
Let a non-deterministic finite automaton be →

Q = {a, b, c}
∑ = {0, 1}
q_0 = {a}
F = {c}

The transition function δ as shown below −

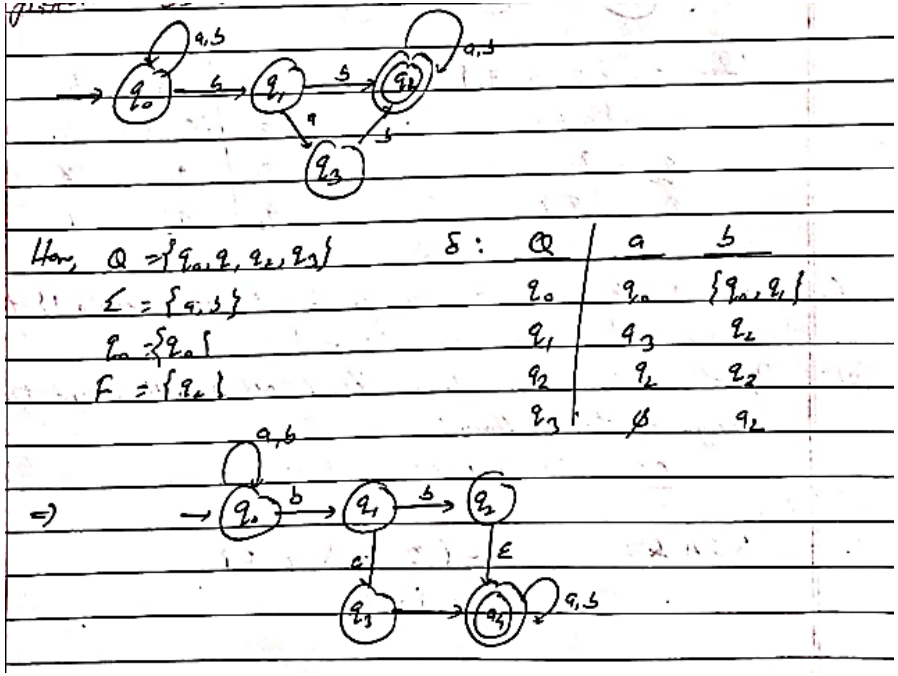
Present State	Next State for Input 0	Next State for Input 1
a	a, b	b
b	c	a, c
c	b, c	c

Its graphical representation would be as follows −

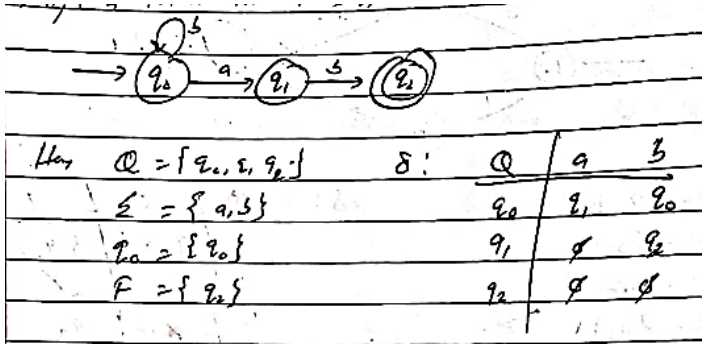


2.6 NFA Numerical Problems

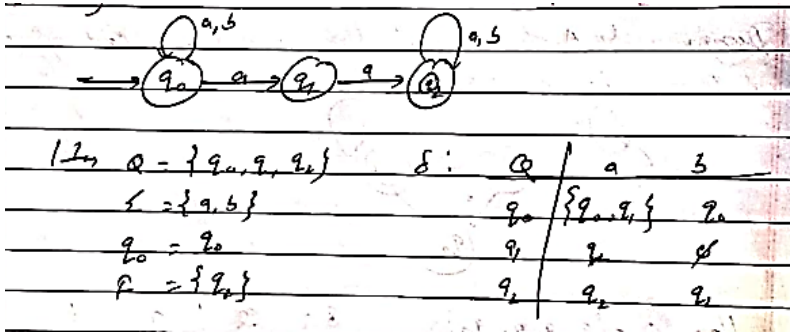
1. **Design a NFA that accepts the set of strings containing occurrence of pattern bb or bab.**



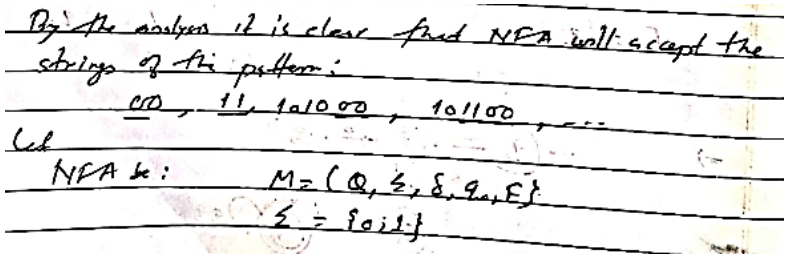
2. Design a NFA for R= b*ab

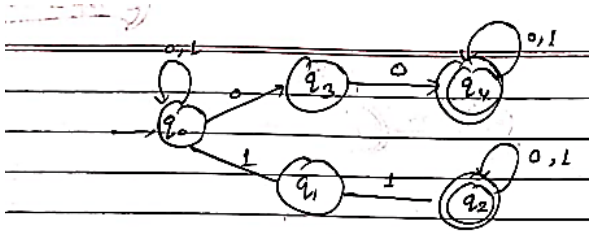


3. Design a NFA over ∑ = {a, b} that accepts strings having aa as substring.

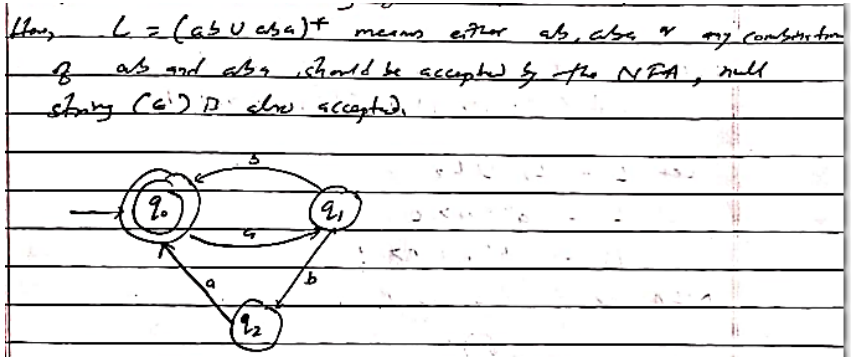


4. Design a NFA for the language L=all strings over {0, 1} that have at least two consecutive 0s and 1s.

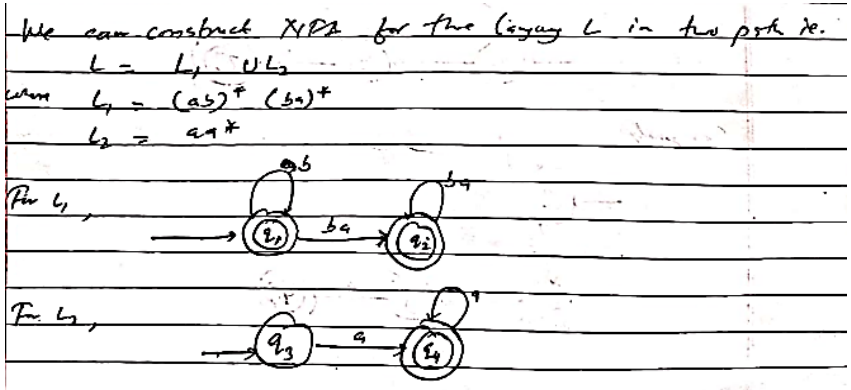


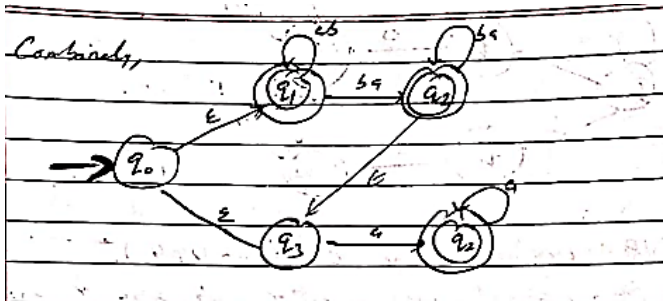


5. Design a NFA for the language L= (ab U aba)*

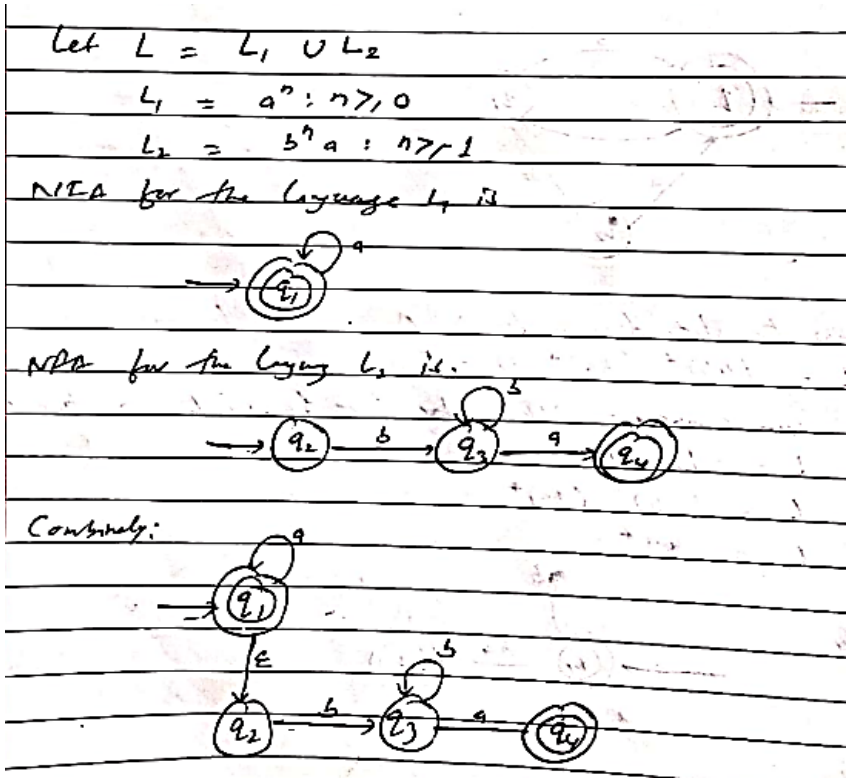


6. Draw the state diagram for NFA accepting the language L= (ab)*(ba)* U aa*





7. **Find NFA with four state for the language L= { (a^n : n ≥ 0) U ($b^n a$: n ≥1)}**



2.7 Equivalence of DFA and NFA

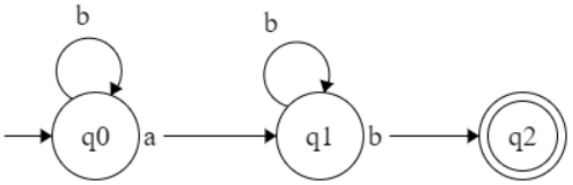
In NFA, when a specific input is given to the current state, the machine goes to multiple states. It can have zero, one or more than one move on a given input symbol. On the other hand, in DFA, when a specific input is given to the current state, the machine goes to only one state. DFA has only one move on a given input symbol. However, for each NFA, there is an equivalent DFA.

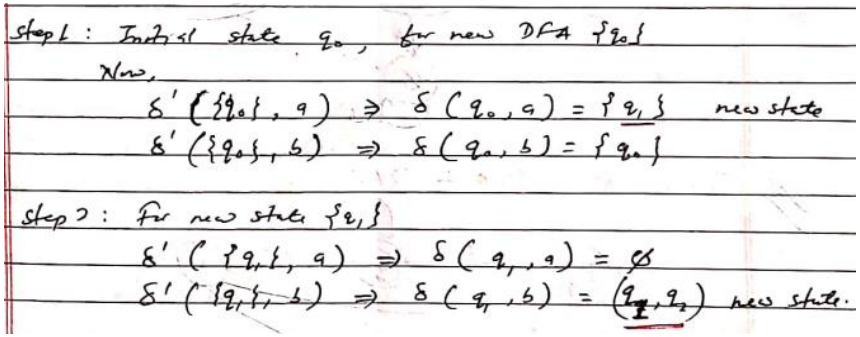
Steps for converting NFA to DFA:

1. Observe all the transitions from starting state q0 for every symbol in alphabet (∑).
2. For new states from step 1, check all transitions for every ∑.
3. Repeat step 2 till new states are appeared.

2.8 NFA to DFA Numerical Problems

1. **Convert below NFA to DFA.**





Step 3: For new state $\{q_1, q_2\}$

$\delta'(\{q_1,q_2\}, a) \Rightarrow \delta(q_1,a) \cup \delta(q_2,a)$

$\Rightarrow \phi \cup \phi$

$\Rightarrow \phi$

$\delta'(\{q_1,q_2\}, b) \Rightarrow \delta(q_1,b) \cup \delta(q_2,b)$

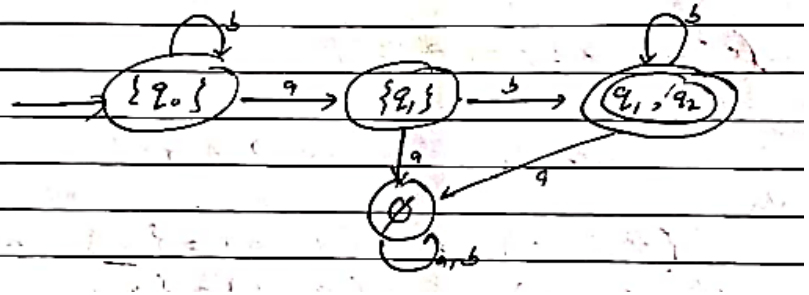
$\Rightarrow \{q_1, q_2\} \cup \phi$

$= \{q_1, q_2\}$

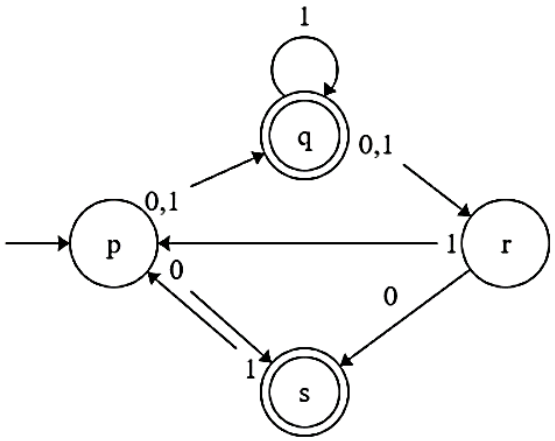
Now, transition table is:

Q	a	b
$\{q_0\}$	$\{q_1\}$	$\{q_0\}$
$\{q_1\}$	ϕ	$\{q_1, q_2\}$
$\{q_1, q_2\}$	ϕ	$\{q_1, q_2\}$

Here, Equivalent DFA is:-



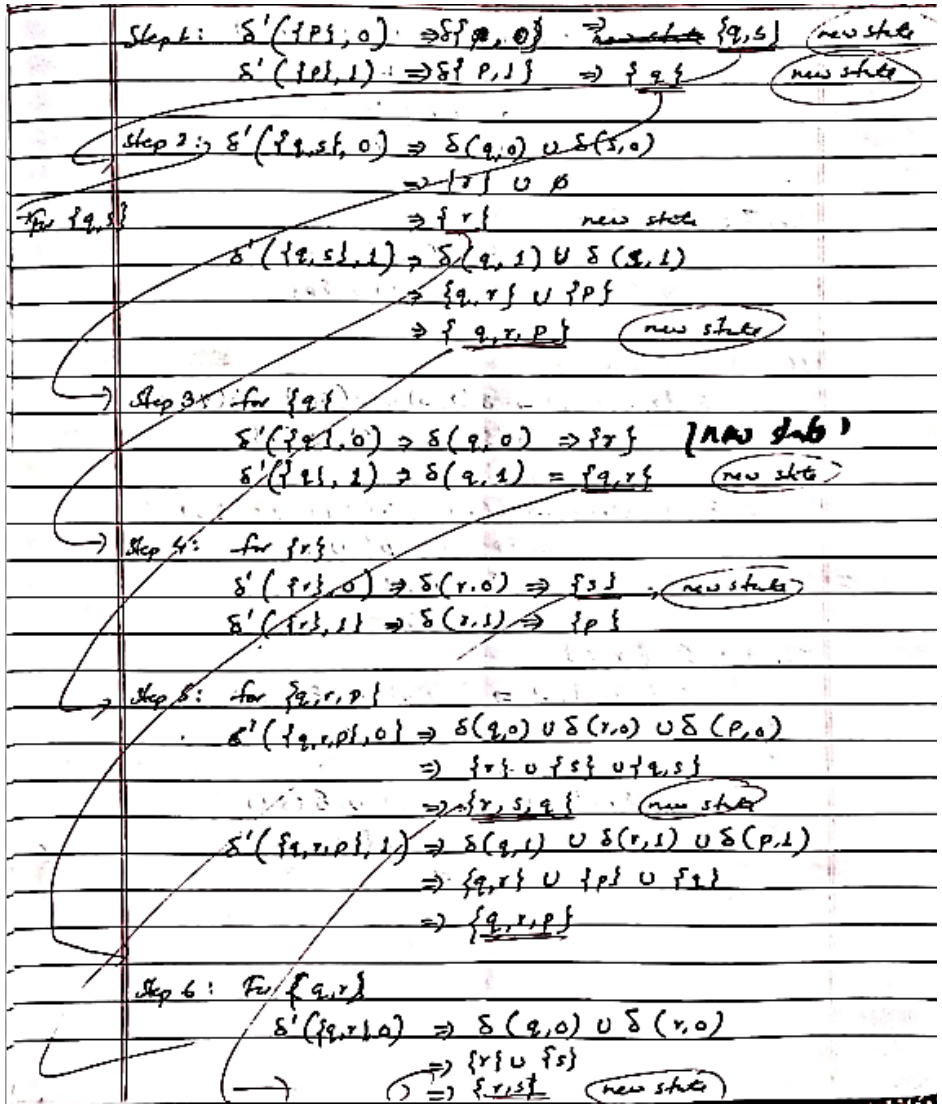
2. Transform below NFA to DFA:

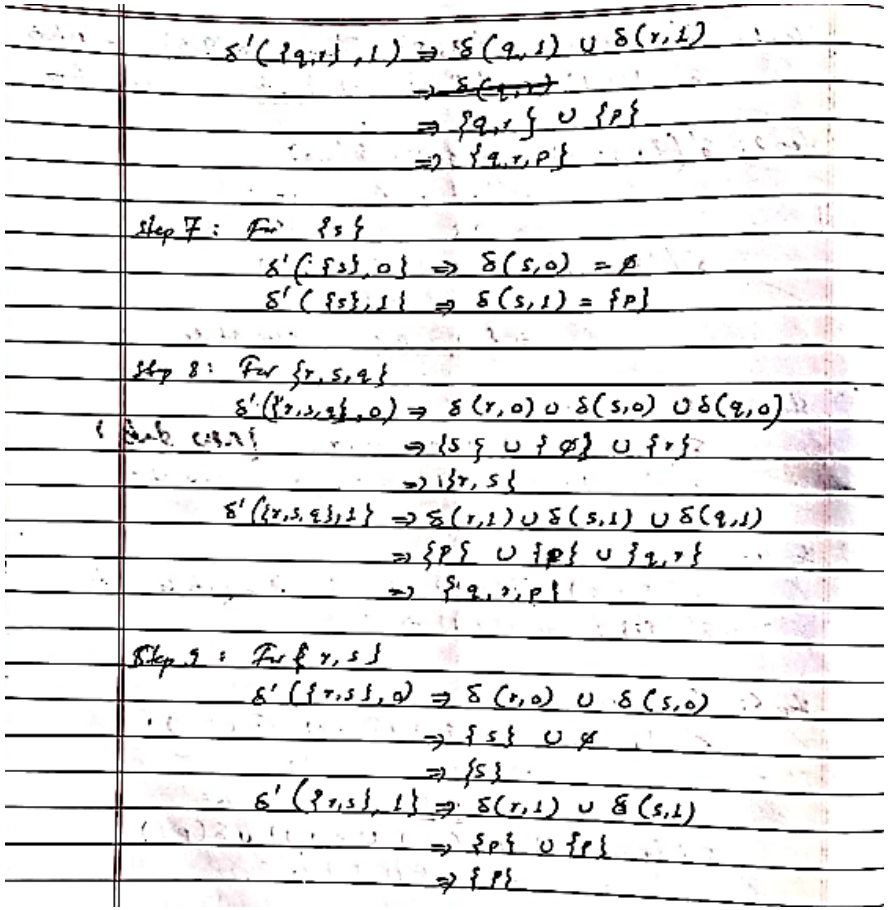


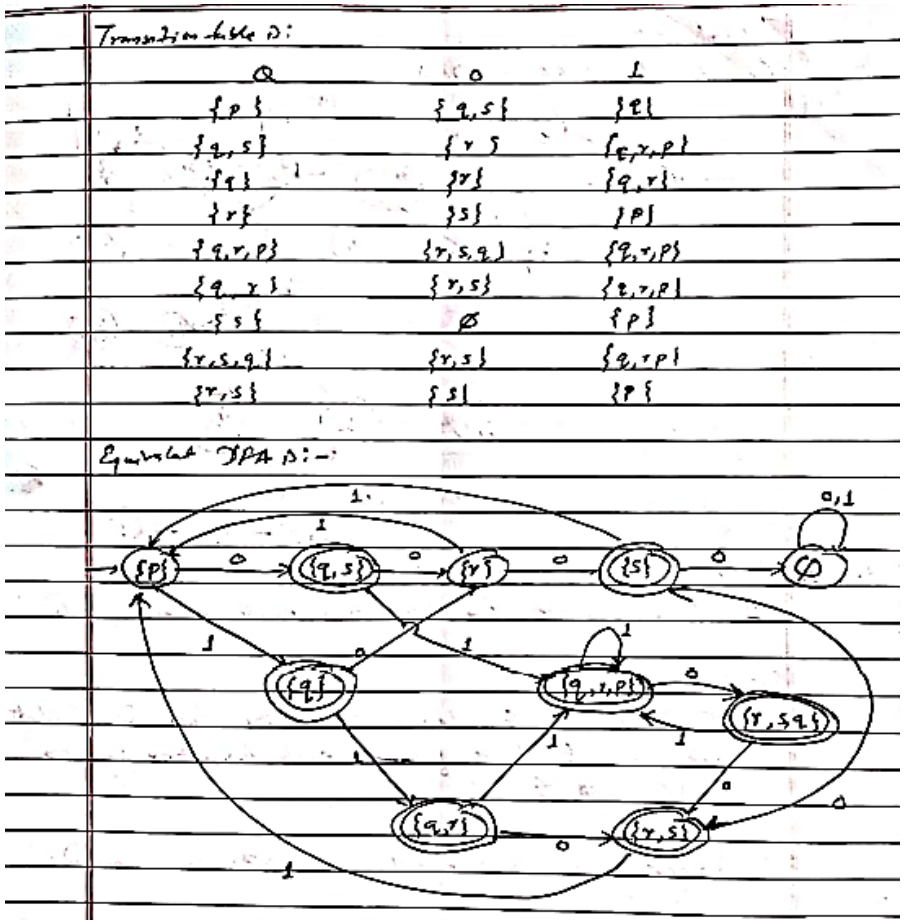
Here, Initial state of NFA is p

For DFA, initial state is {p}

Now,

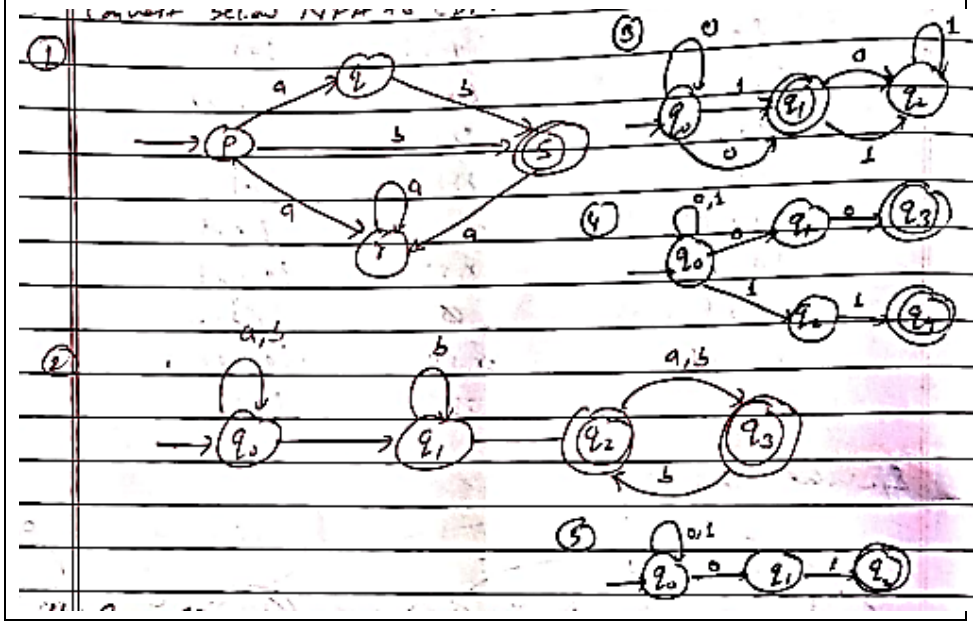

Step 1: δ'({p},0) ⇒ δ{p,0} ⇒ {q,s} new state
δ'({p},1) ⇒ δ{p,1} ⇒ {q} new state
Step 2: δ'({q,s},0) ⇒ δ(q,0) ∪ δ(s,0)
⇒ {r} ∪ ∅
for {q,s} ⇒ {r} new state
δ'({q,s},1) ⇒ δ(q,1) ∪ δ(s,1)
⇒ {q,r} ∪ {p}
⇒ {q,r,p} new state
Step 3: for {q}
δ'({q},0) ⇒ δ(q,0) ⇒ {r} [new state]
δ'({q},1) ⇒ δ(q,1) = {q,r} new state
Step 4: for {r}
δ'({r},0) ⇒ δ(r,0) ⇒ {s} new state
δ'({r},1) ⇒ δ(r,1) ⇒ {p}
Step 5: for {q,r,p}
δ'({q,r,p},0) ⇒ δ(q,0) ∪ δ(r,0) ∪ δ(p,0)
⇒ {r} ∪ {s} ∪ {q,s}
⇒ {r,s,q} new state
δ'({q,r,p},1) ⇒ δ(q,1) ∪ δ(r,1) ∪ δ(p,1)
⇒ {q,r} ∪ {p} ∪ {q}
⇒ {q,r,p}
Step 6: For {q,r}
δ'({q,r},0) ⇒ δ(q,0) ∪ δ(r,0)
⇒ {r} ∪ {s}
⇒ {r,s} new state


δ'({q,r},1) ⇒ δ(q,1) ∪ δ(r,1)
⇒ {q,r} ∪ {p}
⇒ {q,r,p}
Step 7: For {s}
δ'({s},0) ⇒ δ(s,0) = ∅
δ'({s},1) ⇒ δ(s,1) = {p}
Step 8: For {r,s,q}
δ'({r,s,q},0) ⇒ δ(r,0) ∪ δ(s,0) ∪ δ(q,0)
⇒ {s} ∪ {∅} ∪ {r}
⇒ {r,s}
δ'({r,s,q},1) ⇒ δ(r,1) ∪ δ(s,1) ∪ δ(q,1)
⇒ {p} ∪ {p} ∪ {q,r}
⇒ {q,r,p}
Step 9: For {r,s}
δ'({r,s},0) ⇒ δ(r,0) ∪ δ(s,0)
⇒ {s} ∪ ∅
⇒ {s}
δ'({r,s},1) ⇒ δ(r,1) ∪ δ(s,1)
⇒ {p} ∪ {p}
⇒ {p}



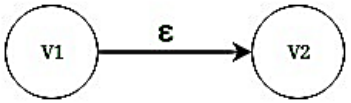
Tutorial:

Convert below NFA to DFA:



2.9 Ɛ -NFA

The NFA with epsilon-transition is a finite state machine in which the transition from one state to another state is allowed without any input symbol i.e. empty string ε. Here in the figure below, State V1 and v2 have an epsilon move.



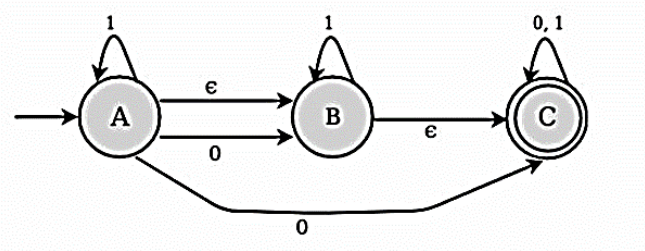
Epsilon (ε) - closure

Epsilon closure for a given state X is a set of states which can be reached from the states X with only (null) or ε moves including the state X itself.

In other words, ε -closure for a state can be obtained by union operation of the ε -closure of the states which can be reached from X with a single ε move in a recursive manner.

Example

Consider the following figure of NFA with ε move −



The transition state table for the above NFA is as follows −

State	0	1	epsilon
A	B, C	A	B
B	-	B	C
C	C	C	-

For the above example, ε closure are as follows −

E closure(A) : {A, B,C}

E closure(B) : {B,C}

E closure(C) : {C}

Converting E-NFA to DFA

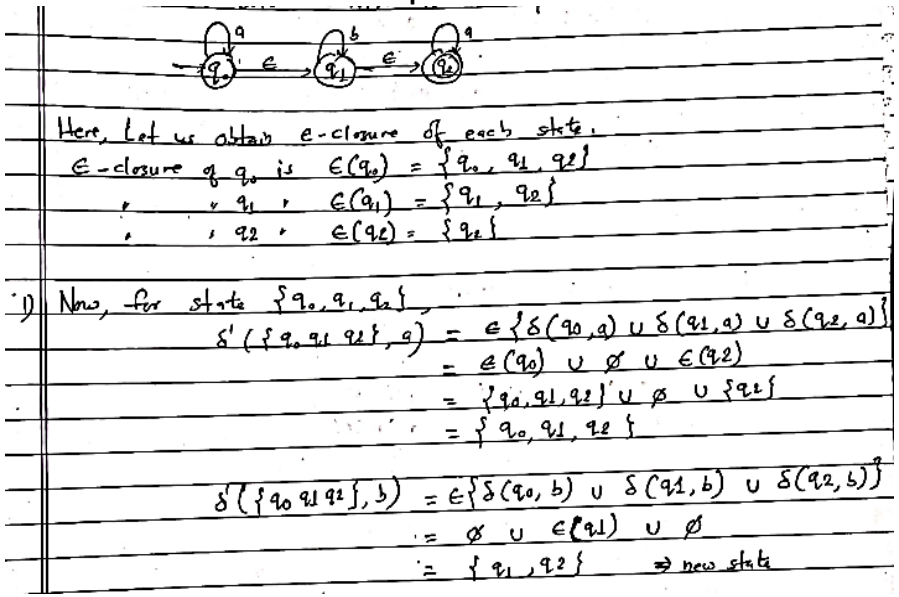
Steps:

1. Find out the ε-closure of starting state, and calculate union of ε-closure of each transition foe every symbol of alphabet (∑).
2. Repeat same process till new states are approved.

Here, ε -closure of state q can be obtained by following all transitions out of q the are labelled ε.

Numerical:

1. **Convert the below ε-NFA to its equivalent DFA.**



2) For state $\{q_1, q_2\}$

$\delta'(\{q_1, q_2\}, a) = \in\{\delta(q_1, a) \cup \delta(q_2, a)\}$
$= \emptyset \cup \in(q_2)$
$= \{q_2\}$ ⇒ new state

$\delta'(\{q_1, q_2\}, b) = \in\{\delta(q_1, b) \cup \delta(q_2, b)\}$
$= \in(q_1) \cup \emptyset$
$= \{q_1, q_2\}$

3) For state $\{q_2\}$:

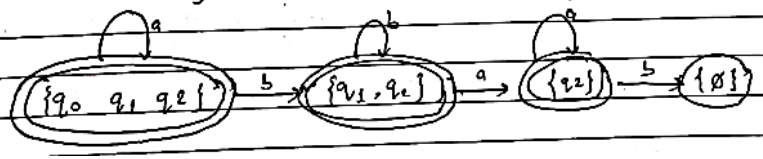
$\delta'(\{q_2\}, a) = \in\{\delta(q_2, a)\}$
$= \in(q_2)$
$= \{q_2\}$

$\delta'(\{q_2\}, b) = \in\{\delta(q_2, b)\}$
$= \emptyset$

Hence, transition table is :-

Q	a	b
$\{q_0 q_1 q_2\}$	$\{q_0 q_1 q_2\}$	$\{q_1, q_2\}$
$\{q_1 q_2\}$	$\{q_2\}$	$\{q_1 q_2\}$
$\{q_3\}$	$\{q_2\}$	$\emptyset$

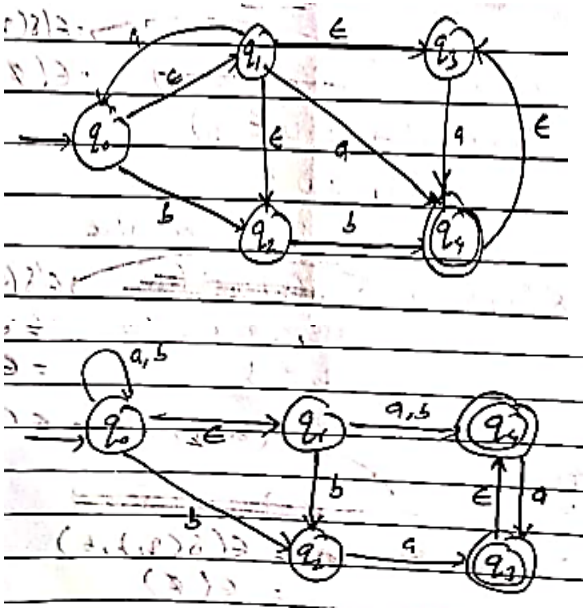
The state diagram of DFA is :-



Tutorial:

Convert below ε-NFA to its equivalent DFA.

1) **2)**

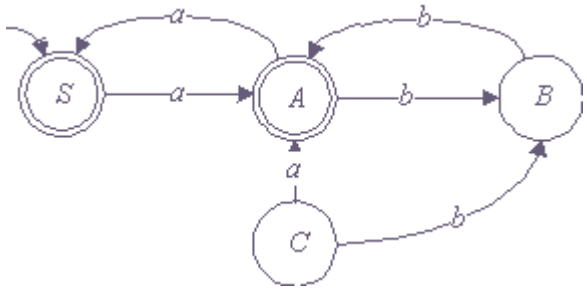


2.10 State Minimization (DFA Minimization)

DFA minimization stands for converting a given DFA to its equivalent DFA with minimum number of states. Minimization of DFA thus reduces the number of states from given FA. Thus, we get the FSM (finite state machine) with redundant states after minimizing the FSM.
In this process we minimize the given DFA by removing unreachable states and by merging similar equivalent states.

Inaccessible states:
All the states which can never be reached from initial state are inaccessible/unreachable states. In contrast, Dead states are the states from which no final state is reachable.

For eg:



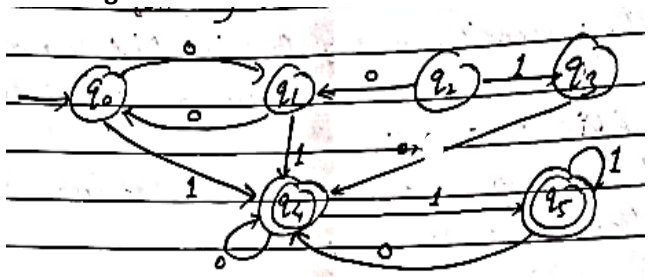
Here, state C is never reached from the initial state S. So state C is unreachable state.

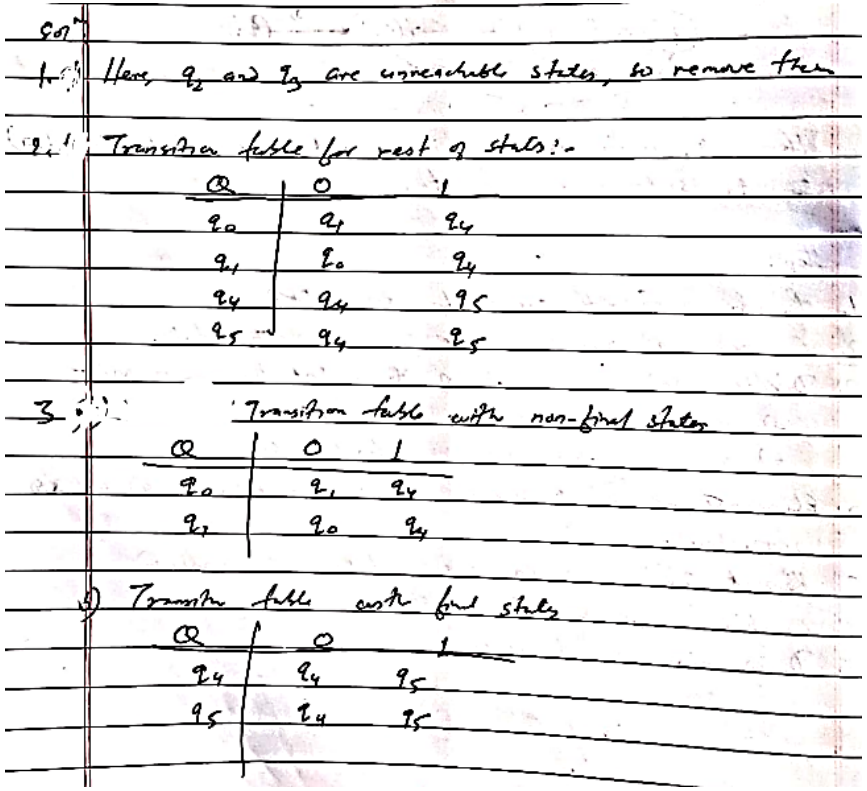
State minimization steps:

1. Remove unreachable states from given DFA.
2. Construct a transition table for rest of the states.
3. Divide the transition table of step 2 into two parts:
   a) Rows that start with final states.
   b) Rows that start with non-final states.
4. Eliminate one of the equivalent/similar states from table : 3a and 3b respectively.
5. Repeat step 4 until equivalent states are appeared.
6. Combine/merge updated tables.
7. Draw minimized state diagram.

2.11 Numerical

1. **Minimize following DFA.**





4. Eliminate equivalent/similar rows in 5th table q_3 and q_5 so, we have. 3(5) as: $(q_4 = q_5)$.

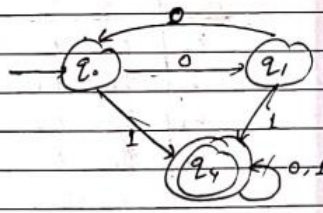
Q	0	1
q_3	q_4	q_4

5. Merge updated table: we get.

Q	0	1
q_0	q_1	q_4
q_1	q_0	q_4
q_4	q_4	q_4

$\therefore (q_4 = q_5)$

6. Final minimized DFA D:-

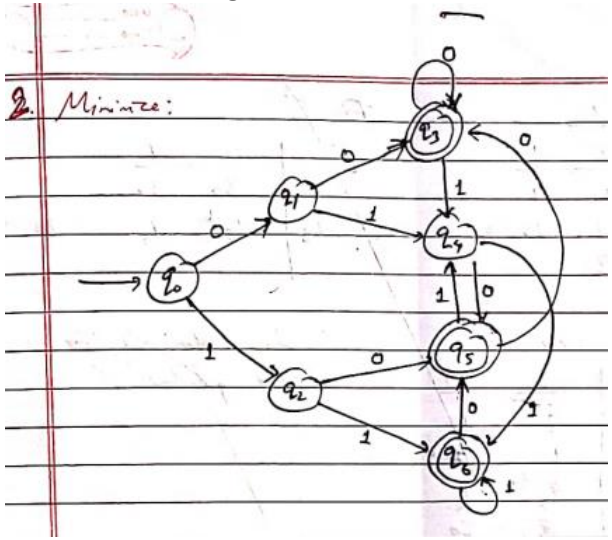


Test: 00100 1

$\Rightarrow \delta(q_0, 00100 1)$
$\vdash (q_1, 0100 1)$
$\vdash (q_0, 100 1)$
$\vdash (q_4, 00 1)$
$\vdash (q_4, 01)$
$\vdash (q_4, 1)$
$\vdash (q_4, \epsilon)$

$\therefore Q_4$ is in final state, so DFA accepted.

2. Minimize following DFA



Solution:

1. No unreachable states.
2. Transition table:

Q	0	1
q_0	q_1	q_2
q_1	q_3	q_4
q_2	q_5	q_6
q_3	q_3	q_4
q_4	q_5	q_6
q_5	q_3	q_4
q_6	q_5	q_6

3. Splitting into transition tables of Non-final and final states,

a) Non-final state Transition table

Q	0	1
q_0	q_1	q_2
q_1	q_3	q_4
q_2	q_5	q_6
q_4	q_5	q_6

b) Final state Transition table

Q	0	1
q_3	q_3	q_4
q_5	q_3	q_4
q_6	q_5	q_6

4. Eliminating one of the equivalent rows.

a)

Q	0	1
q_0	q_1	q_2
q_1	q_3	q_4
q_2	q_5	q_6
~~q_4~~	~~q_5~~	~~q_6~~

Here, Now, $q_2 = q_4$

⇓

Q	0	1	
q_0	q_1	q_2	
q_1	q_3	q_2	$\because q_4 = q_2$
q_2	q_3	q_6	$\because q_5 = q_3$

b)

Q	0	1
q_3	q_3	q_4
~~q_5~~	~~q_3~~	~~q_4~~
q_6	q_5	q_6

Here, Now, $q_3 = q_5$

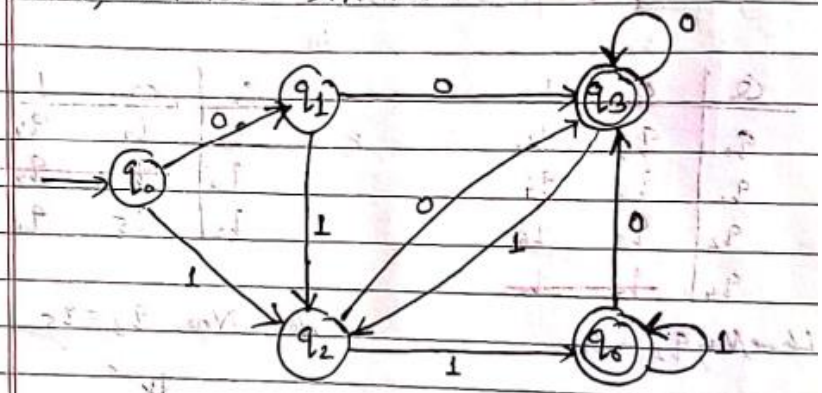
⇓

Q	0	1	
q_3	q_3	q_2	$\because q_4 = q_2$
q_6	q_3	q_6	$\because q_5 = q_3$

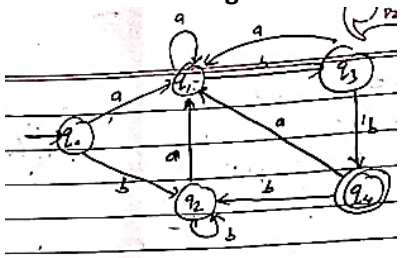
5. Merging the tables,

Q	0	1
q_0	q_1	q_2
q_1	q_3	q_2
q_2	q_3	q_6
q_3	q_3	q_2
q_6	q_3	q_6

6. Hence, minimized DFA is:-



3. Minimize following DFA.



Solution:

1. No unreachable states
2. Transition table:

Q	a	b
q_0	q_1	q_2
q_1	q_1	q_3
q_2	q_1	q_2
q_3	q_1	q_4
q_4	q_1	q_2

3. a) Non-final

Q	a	b
q_0	q_1	q_2
q_1	q_1	q_3
q_2	q_1	q_2
q_3	q_1	q_4

b) Final

Q	a	b
q_4	q_1	q_2

4. Eliminate equivalent row $q_0 \equiv q_2$

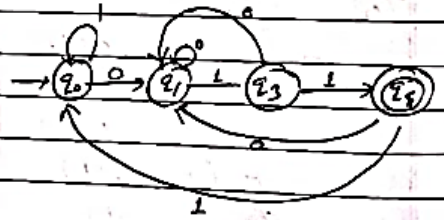
Q	a	b
q_0	q_1	q_0

5. Merge table:

Q	a	b
q_0	q_1	q_0
q_1	q_1	q_3
q_3	q_1	q_4
q_4	q_1	q_0

$\therefore q_0 = q_2$

6. State diagram:

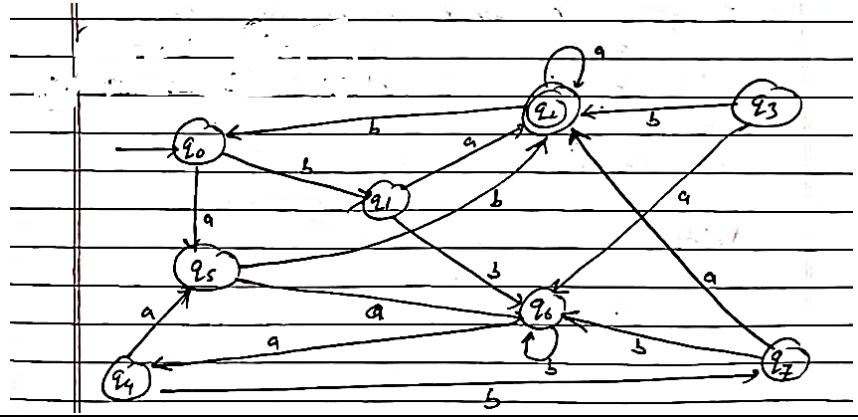


Tutorial:

1. **Minimize following DFA by using State Minimization method, where: → represents initial state and * represents final state.**

S/Σ	0	1
→ q_0	q_1	q_2
* q_1	q_1	q_3
q_2	q_2	q_2
* q_3	q_5	q_2
* q_4	q_4	q_2
* q_5	q_4	q_2
q_6	q_5	q_6
q_7	q_5	q_6

2. **Minimize below DFA:**



2.12 Regular Expressions and Regular Language

Regular Expression

- The language accepted by finite automata can be easily described by simple expressions called Regular Expressions. It is the most effective way to represent any language.
- The languages accepted by some regular expression are referred to as Regular languages.
- A regular expression can also be described as a sequence of pattern that defines a string.
- Regular expressions are used to match character combinations in strings. String searching algorithm used this pattern to find the operations on a string.
- For instance: In a regular expression, x* means zero or more occurrence of x. It can generate {e, x, xx, xxx, xxxx, ...}
- In a regular expression, x+ means one or more occurrence of x. It can generate {x, xx, xxx, xxxx, ...}

Definition of Regular Expression:

The set of regular expression is defined by the following rules"

1. Every letter of ∑ can be made into a regular expression, null string, ε itself is a regular expression.
2. If r1 and r2 are regular expression, then:
   i) (r1)
   ii) r1r2
   iii) r1 + r2
   iv) r1*
   v) r1+ are also regular expression.
3. Nothing else is regular expression.

Regular Language:

A language is regular if it can be expressed in terms of regular expression.

Regular Expression	Regular language
a	{a}
B	{b}
a+b	{a,b}
a··b	{ab}
a∗a∗	{ϵϵ,a,aa,aaa,aaaa,.....}
b+b+	{b,bb,bbb,bbbb,.....}

2.13 Inductive Steps

There are four parts to the inductive step, one for each of three operators and one for the introduction of parentheses.

1. If r1 and r2 are regular expressions, then r1 +r2 is a regular expression denoting the union of L(r1) and L(r2).
   i.e. L(r1+r2)= L(r1) U L(r2)
2. If r1 and r2 are regular expressions, then r1 r2 is a regular expression denoting the concatenation of L(r1) and L(r2).
   i.e. L(r1r2)= L(r1) . L(r2)
3. If r is a regular expressions, then r* is a regular expression denoting the closure of L(r).
   i.e. L(r*)= (L(r))*
4. If r is a regular expressions, then (r) , a parenthesized r, is also a regular expression denoting the same language as r.
   i.e. L((r))= L(r)

2.14 Application of Regular language

1. Validation:	Determines that a string compiles a set of formatting constraints like email validation, password validation, etc.
2. Search and Selection:	Identifying a subset of items from larger set on the basis of a pattern match
3. Tokenization:	Converting a sequence of characters into words, tokens for later interpretation.

2.15 Algebraic laws for RE

If r, r1, r2 and r3 are regular expressions, then:

Commutativity Law	• r1 U r2 = r2 U r1 • r1.r2 ≠ r2. r1
Associativity Law	• r1 U (r2 U r3) = (r1 U r2) U r3 • r1.(r2 .r3) = (r1. r2). r3
Distributive Law	• r1 (r2 U r3) =r1 r2 U r1 r3 • (r2 U r3) r1 = r2r1 U r3r1
Identity Law	• r1 U Φ = Φ U r1 = r1 • r1. ε = ε. r1 = r1
Annihilator Law	• Φ. r= r. Φ = Φ
Idempotent Law	• r U r =r

2.16 Finite Automata and RE

We can convert each of regular expression into finite automata. Also, from given automata, we can find out the regular expressions.

Conversion of RE to FA

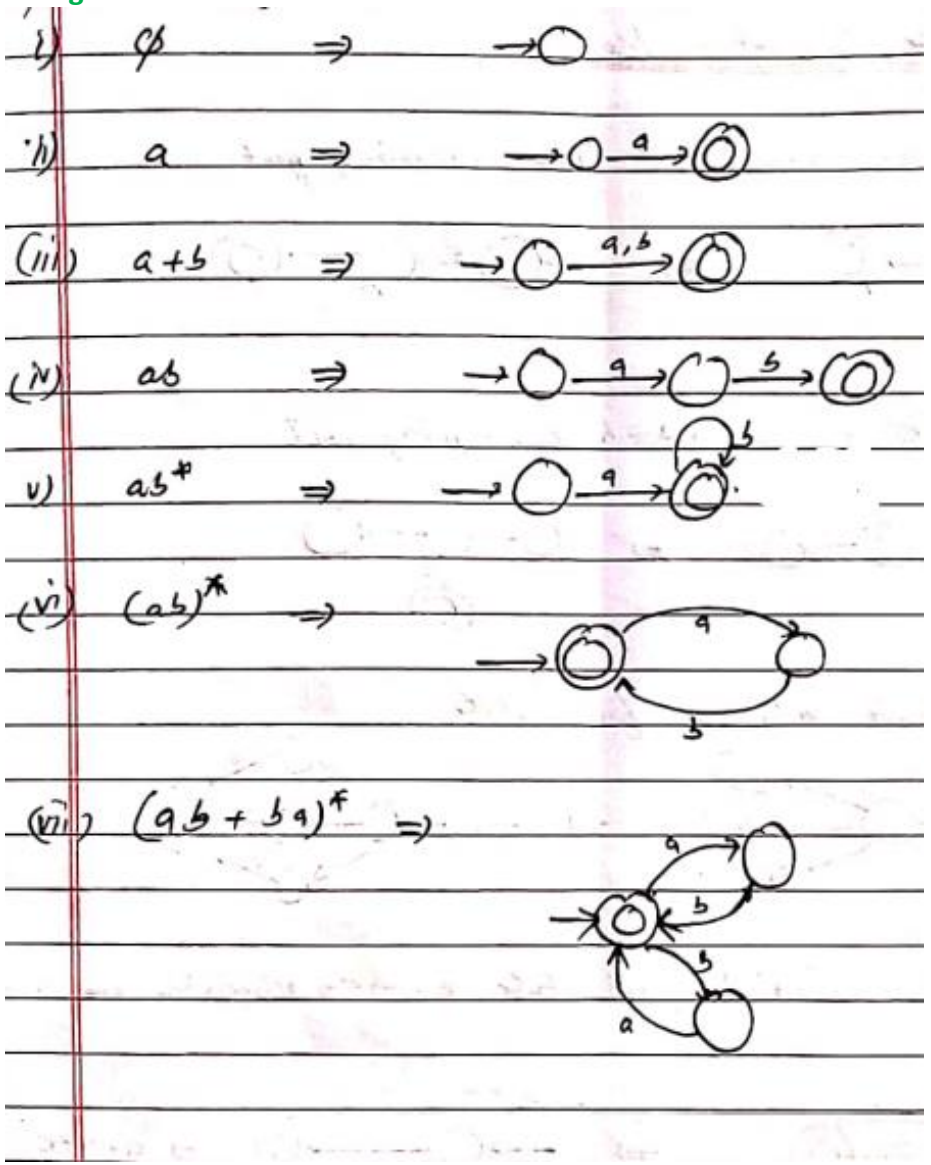
To convert the RE to FA, we can use a method called the **subset** method. This method is used to obtain FA from the given regular expression. This method is given below:

Step 1: Design a transition diagram for given regular expression, using NFA with ε moves.

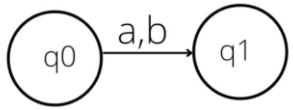
Step 2: Convert this NFA with ε to NFA without ε.

Step 3: Convert the obtained NFA to equivalent DFA.

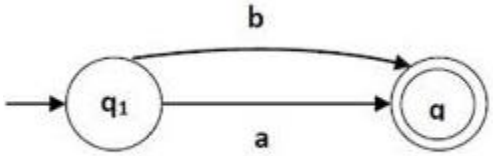
From Regex to FA



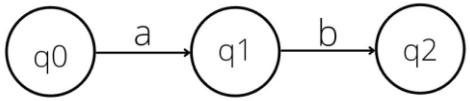
1.) If RE is in the form **a+b,** it can be represented as:



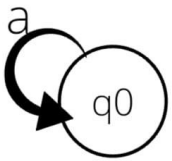
Or



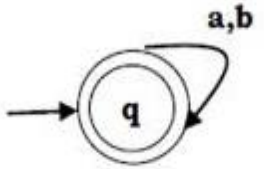
2.) If RE is in the form **ab,** it can be represented as:



3.) If RE is in the form of **a*,** it can be represented as:



4.) For a RE (a+b)*, We can construct FA as mentioned below −

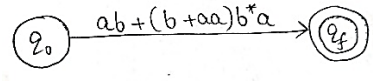


Examples

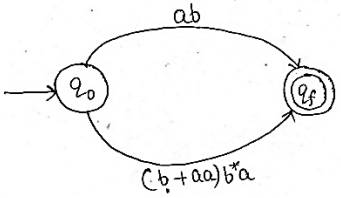
Example 1: Design a Finite Automata from the given RE [ab + (b + aa)b* a].

Solution. At first, we will design the Transition diagram for the given expression.

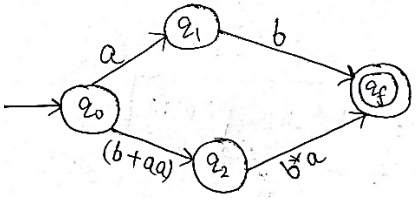
Step 1:



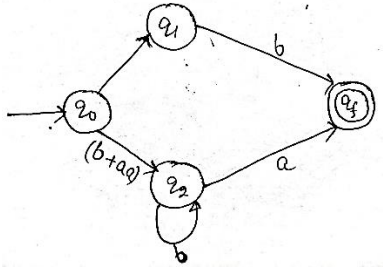
Step 2:



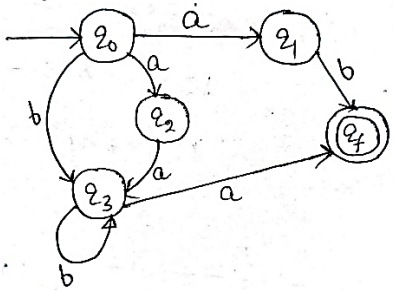
Step 3:



Step 4:



Step 5:



After step 5, we have got NFA without ε. Now we will convert it into the required DFA; we will first write a transition table for this NFA.

Transition Table For NFA:

State	a	b
→q0	{q1,q2}	q3
q1	ϕ	qf
q2	q3	ϕ
q3	qf	q3
*qf	ϕ	ϕ

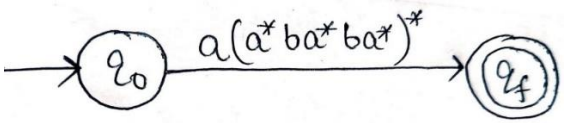
The Corresponding transition table for DFA is:

State	a	b
→q0	[q1,q2]	q3
q1	ϕ	qf
q2	q3	ϕ
q3	qf	q3
[q1,q2]	qf	qf
*qf	ϕ	ϕ

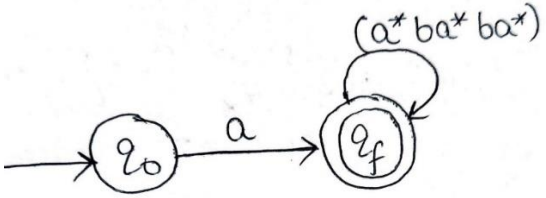
Example 2: Design an NFA from the given RE **[a (a* ba* ba*)*]**.

Solution. The Transition diagram that represents the NFA of the above expression is as follow:

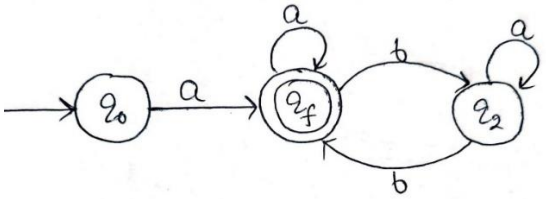
Step 1:



Step 2:



Step 3:



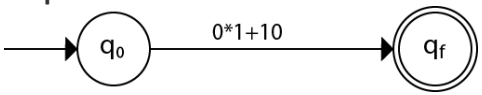
The above is the required NFA.

Example 3:

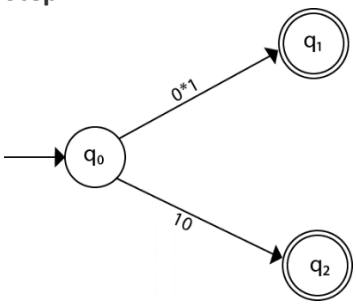
Construct the FA for regular expression 0*1 + 10.

We will first construct FA for R = 0*1 + 10 as follows:

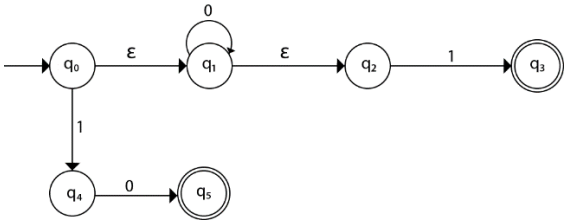
Step 1:



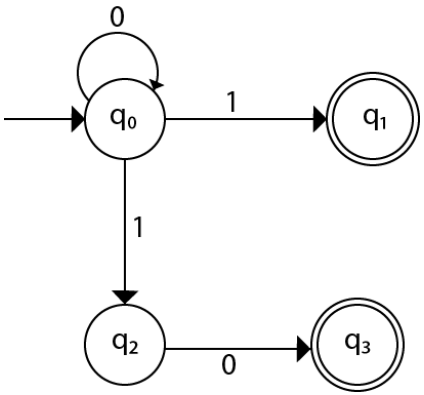
Step 2:



Step 3:

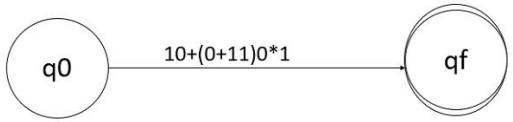


Step 4:



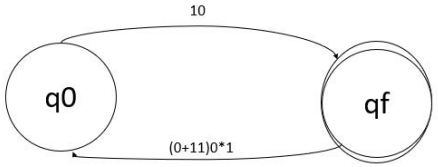
Example 4: Construct finite automata for this regular expression RE 10+(0+11)0*,

Step 1



Initially consider two states q0 and qf. Also, consider the initial and final states, q0 to qf and the given regular expression 10+(0+11)0*1

Step 2



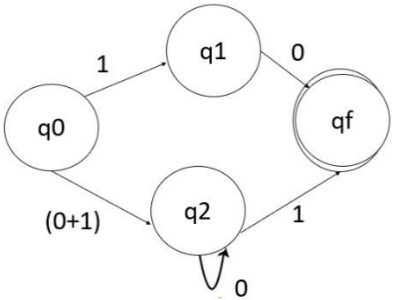
Now applying the union operation, q0 on 10 goes to qf and q0 on (0+11)0*1 goes to qf.

Step 3

Now, divide the expression by applying concatenation i.e. let L1=0, L2=1. Concatenate both and we get L= L1.L2.

Applying this formula on regular expression, so q0 on 1 goes to q1 and q1 on 0 goes to qf. Similarly, q0 on (0+1) goes to q2 and q2 on 0*1 goes to qf.

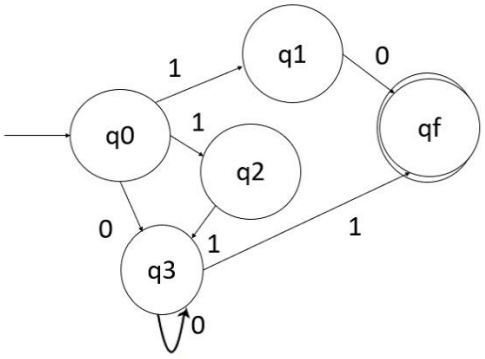
Step 4



q0 on 1 goe to q1 and q1 on 0 goes to qf, q0 on (0+1) goes to q2, q2 on 0 goes to q2 itself and on 1 goes to qf.

Step 5

The final finite automata for the given regular expression is as follows –

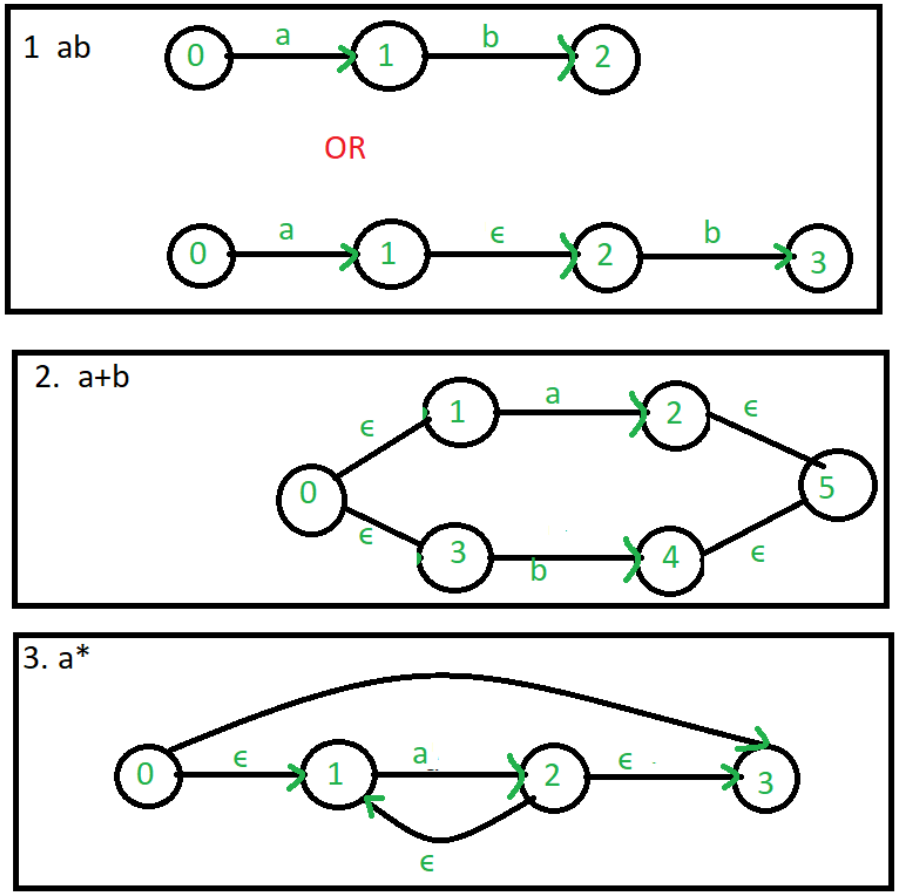


Regular expression to ∈-NFA

∈-NFA is similar to the NFA but have minor difference by epsilon move. This automaton replaces the transition function with the one that allows the empty string ∈ as a possible input. The transitions without consuming an input symbol are called ∈-transitions.

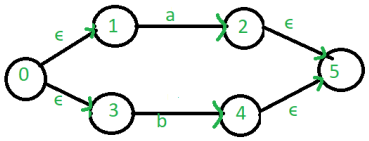
In the state diagrams, they are usually labeled with the Greek letter ∈. ∈-transitions provide a convenient way of modeling the systems whose current states are not precisely known: i.e., if we are modeling asystem and it is not clear whether the current state (after processing some input string) should be q or q', then we can add an ∈-transition between these two states, thus putting the automaton in both states simultaneously.

Common regular expression used in make ∈-NFA:

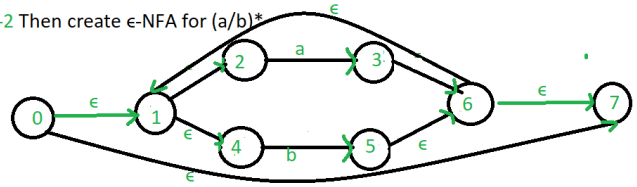


Example: Create a ∈-NFA for regular expression: (a/b)*a

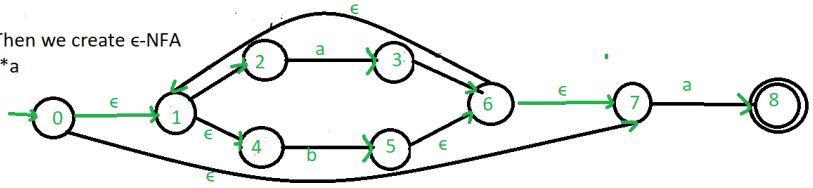
Step-1 First we create ∈-NFA for (a/b)



Step-2 Then create ∈-NFA for (a/b)*



Step-3 Then we create ∈-NFA for(a/b)*a



Example 4.12. One the basis of above discussion find the automation for regular expression $a \,.\, (a + b)^* \,.\, b \,.\, b$.

Solution. Let us solve this problem step by step :

Step 1. The basic regular expression involved are a and b, we start with automation for a and automation for b. Since brackets are evaluated first. We first constructed automation for $(a + b)$, using automation for a and automation b shown in Fig. 4.8(a)

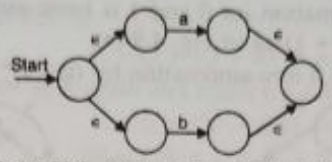


Fig. 4.8(a). Automation for $(a + b)$.

Step 2. Since closure is required to take next, we construct automation for $(a + b)^*$ using automation for $(a + b)$ as shown in Fig. 4.8(b).

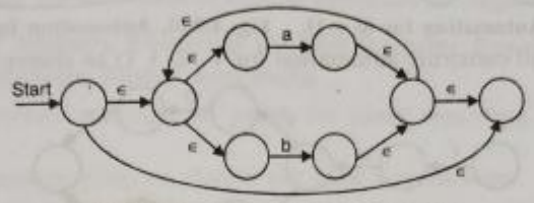


Fig. 4.8(b). Automation for $(a + b)^*$

Step 3. Next we construct the automation for $a \,.\, (a + b)^*$ as shown in Fig. 4.8 (c).

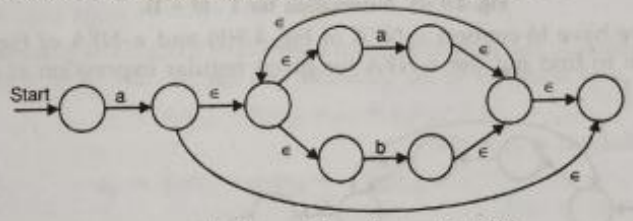


Fig. 4.8(c). Automation for $a \,.\, (a + b)^*$.

Fig. 4.8(c). Automation for $a \cdot (a + b)^* \cdot b$

Step 4. Next we construct the automation for $a \cdot (a + b)^* \cdot b$ by using automation for $a \cdot (a + b)^*$.

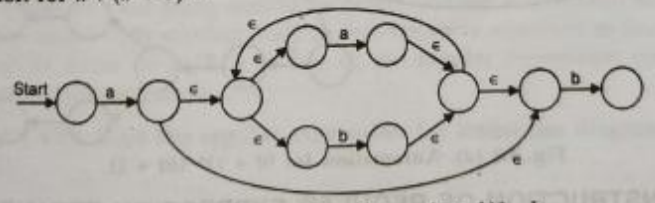


Fig. 4.8(d). Automation for $a \cdot (a + b)^* \cdot b$.

Step 5. Now finally we can construct automation for $a \cdot (a + b)^* \cdot b \cdot b$ by using automation for $a \cdot (a + b)^* \cdot b$.

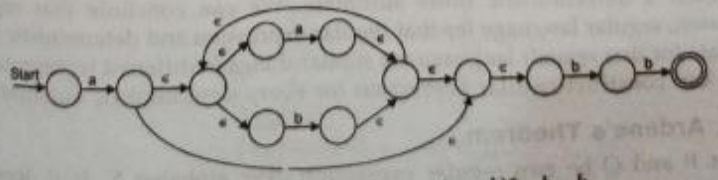


Fig. 4.8(e). Automation for $a \cdot (a + b)^* \cdot b \cdot b$.

It is required automation for given regular expression.

Generating Regular Expression from Finite Automata

There are two popular methods for converting a DFA to its regular expression −

- State elimination method
- Arden's Method

1. State Elimination Method:

Rules

The rules for state elimination method are as follows −

Rule 1

The initial state of DFA must not have any incoming edge.

If there is any incoming edge to the initial edge, then create a new initial state having no incoming edge to it.

Rule 2

There must exist only one final state in DFA.

If there exist multiple final states, then convert all the final states into non-final states and create a new single final state.

Rule 3

The final state of DFA must not have any outgoing edge.

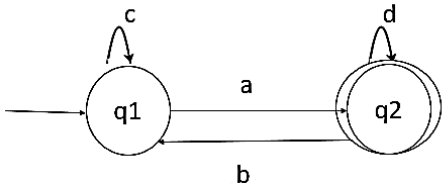
If this exists, then create a new final state having no outgoing edge

Rule 4

Eliminate all intermediate states one by one.

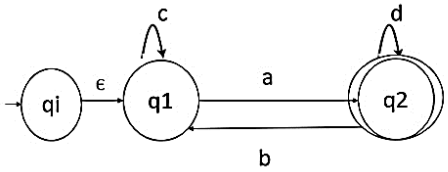
Example 1:

Convert the below FA to RE:



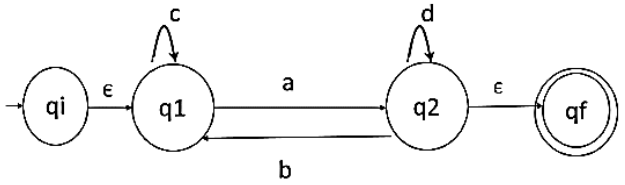
Step 1

Initial state q1 has an incoming edge so create a new initial state qi.



Step 2

Final state q2 has an outgoing edge. So, create a new final state qf.



Step 3

Start eliminating intermediate states

First eliminate q1

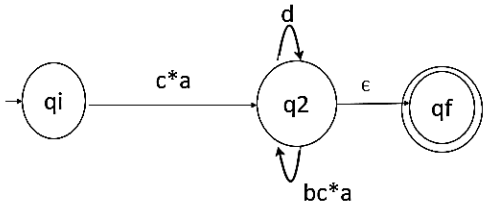
There is a path going from qi to q2 via q1. So, after eliminating q1 we can connect a direct path from qi to q2 having cost.

εc*a=c*a

There is a loop on q2 using state qi. So, after eliminating q1 we put a direct loop to q2 having cost.

b.c*.a=bc*a

After eliminating q1, the FA looks like following −



Second eliminate q2

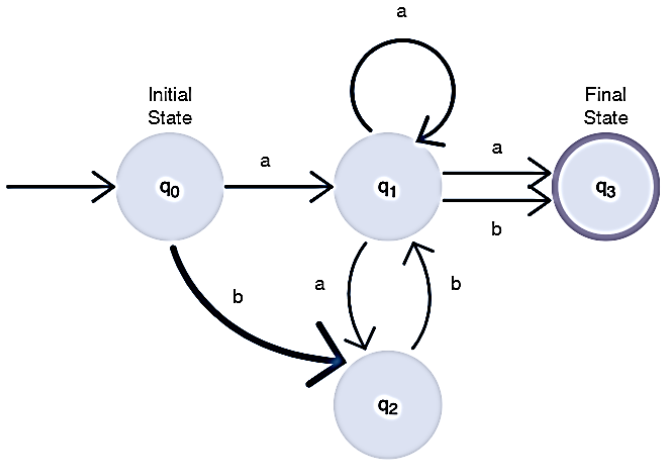
There is a direct path from qi to qf so, we can directly eliminate q2 having cost −

c*a(d+bc*a)* ε = c*a(d+bc*a)*

Which is our final regular expression for given finite automata.

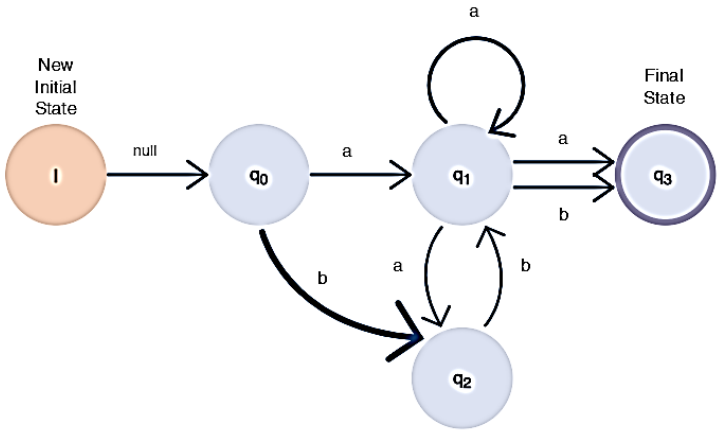
Example 2

Convert below finite automaton to RE.



Step 1: Add an initial state

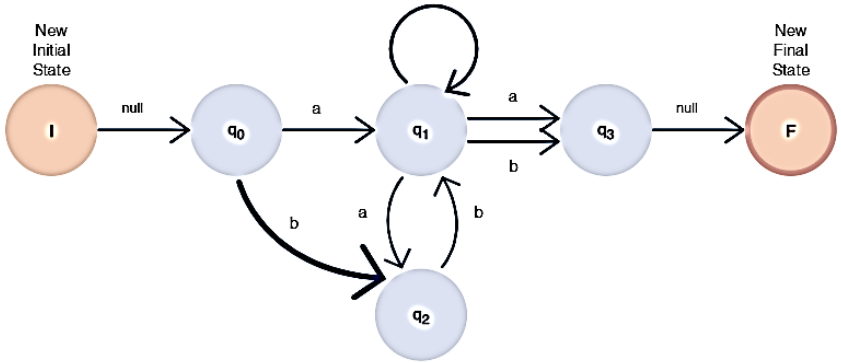
Add a new initial state, I. Make a null transition from state I to state *q*0.



Adding state I

Step 2: Add a final state

Add a new final state, F. Make a null transition from state *q*3 to state F.



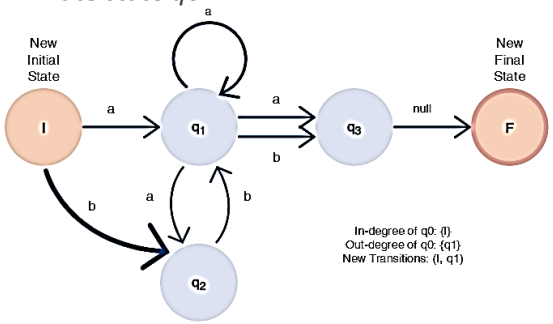
Adding state F

Step 3: State elimination

Perform the elimination of states other than I and *F*.

Step 3.1

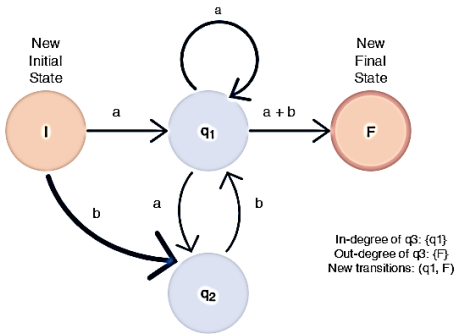
Eliminate state *q*0.



Eliminating q0

Step 3.2

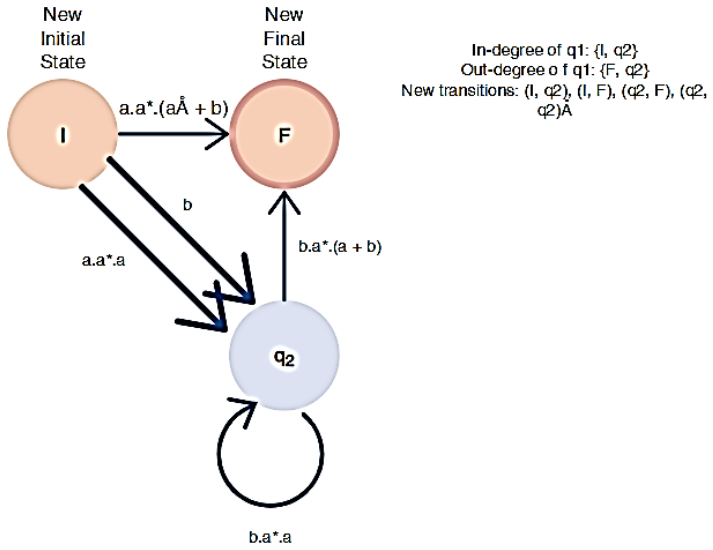
Eliminate state *q*3. Concatenate transitions from state *q*3 to state *F* as per the basic rules of writing regular expressions.



Eliminating q3

Step 3.3

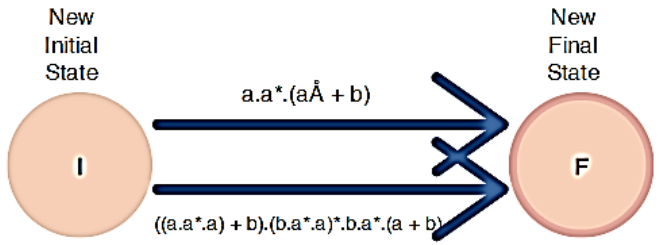
Eliminate *q*1. Check for the in-degree and out-degree of state *q*1. Write the regular expressions for the new transitions acquired after removing state *q*1.



Eliminating q1

Step 3.4

Eliminate state *q*2.

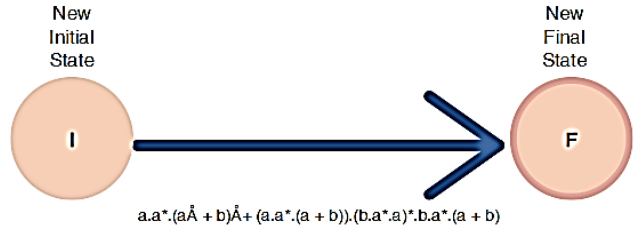


In-degree of q2: {I}
Out-degree of q2: {F}
New Transitions: (I, F)

Eliminating q2

Step 3.5

Put it all together.



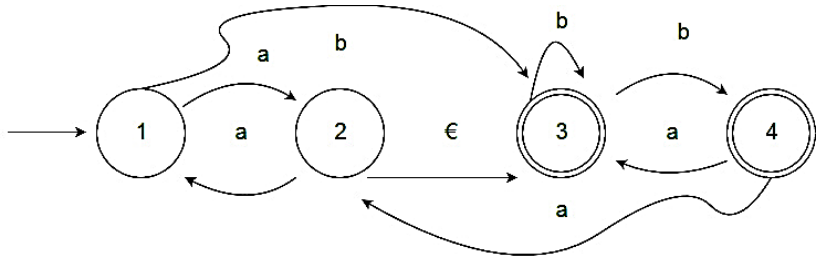
The final regular expression

Result

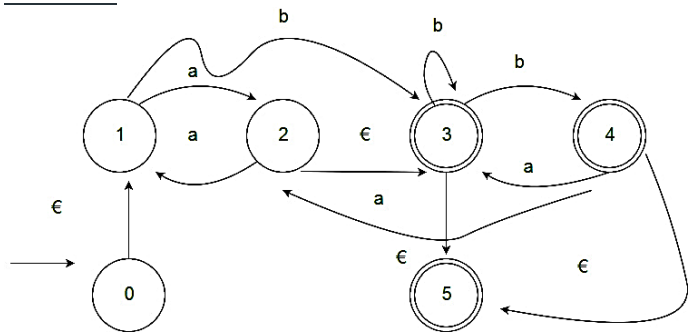
The resultant regular expression for the given finite automaton is as follows:
a.a*.(a + b) + ((a.a*.a) + b).(b.a*.a)*.b.a*.(a + b)*a.a∗.(a+b)+((a.a∗.a)+b).(b.a∗.a)∗.b.a∗.(a+b).

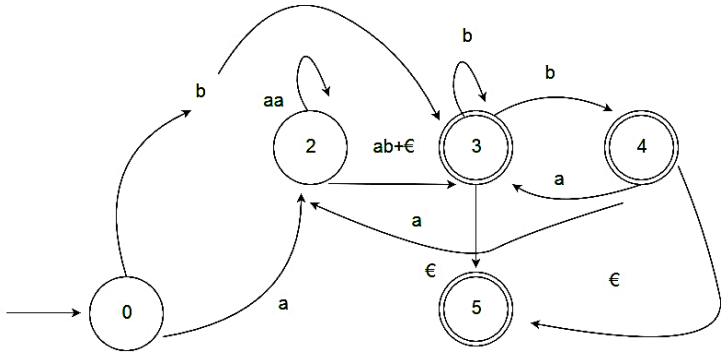
Example 3:

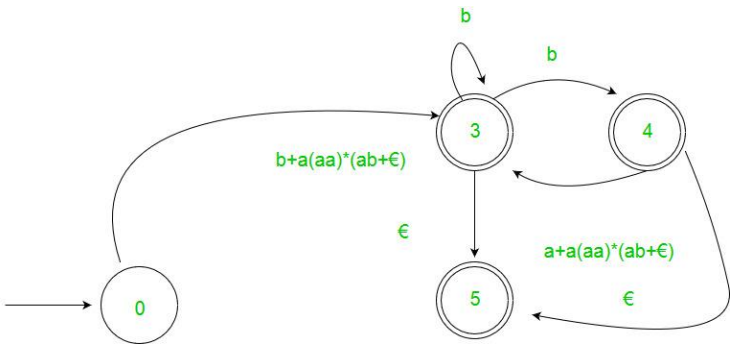
Deduce the RE from below FA.

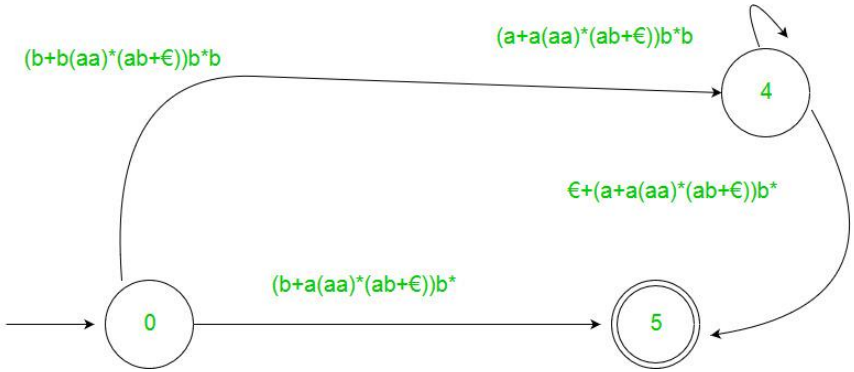


Solution:

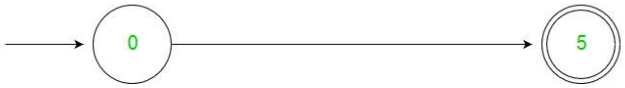




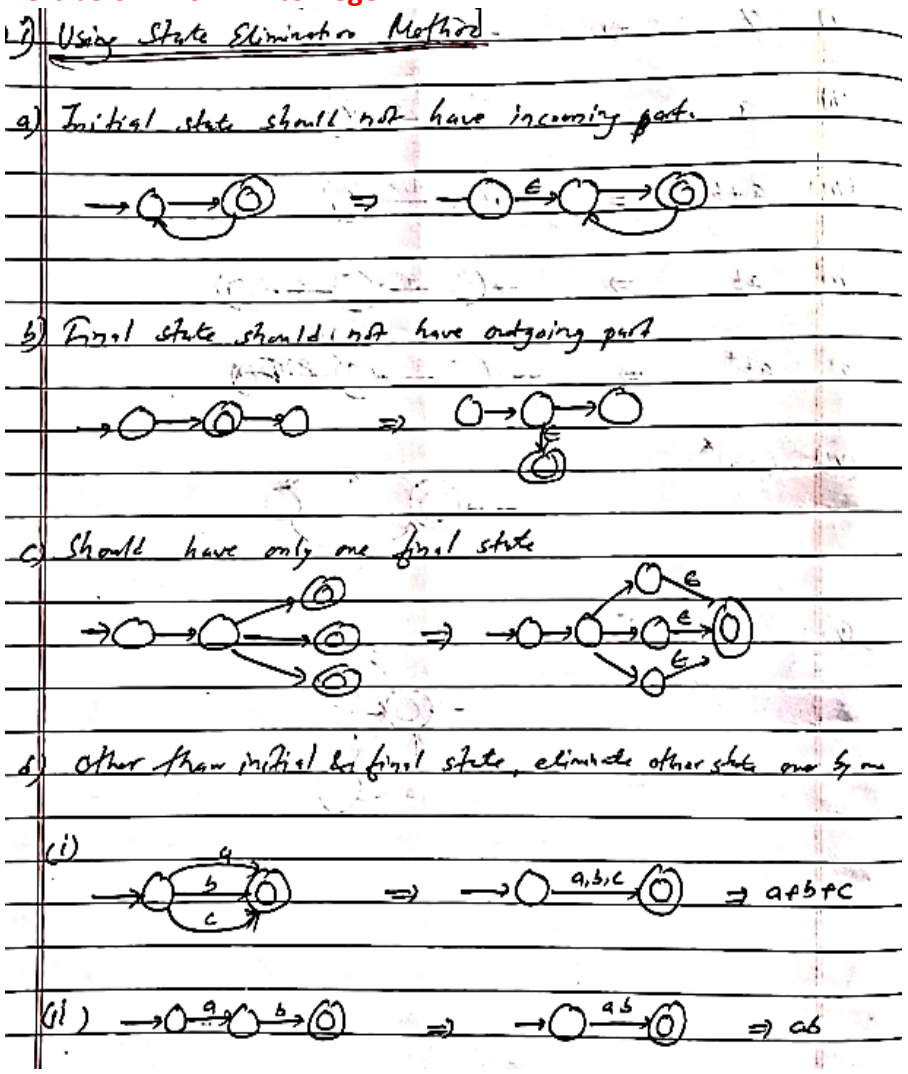


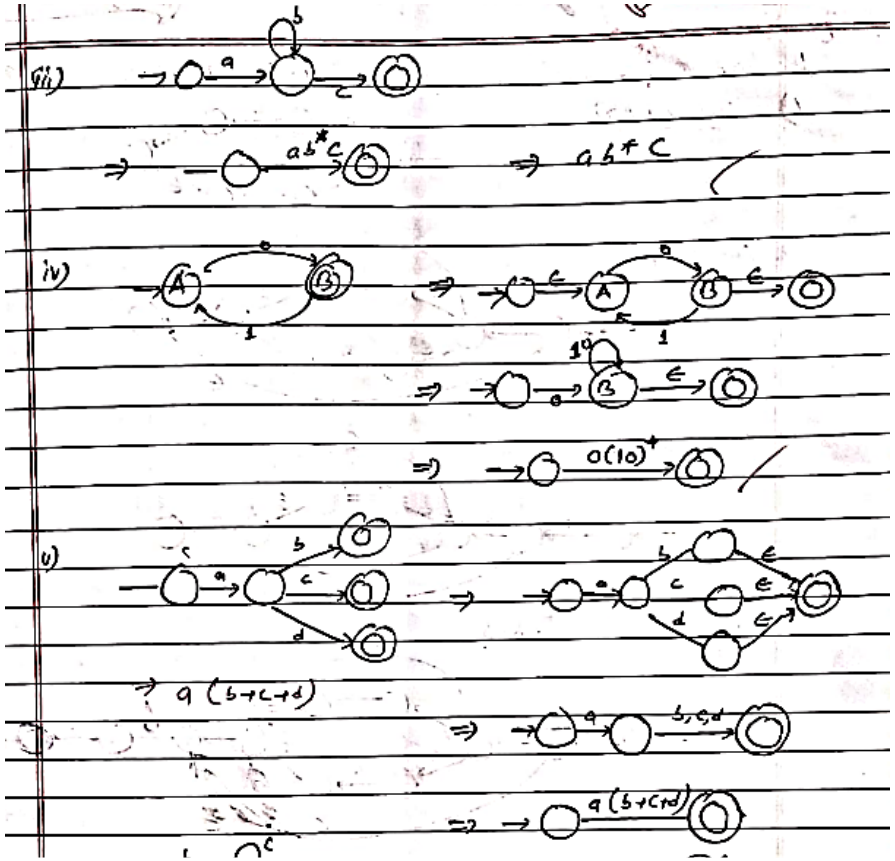


(b+a((aa)*(ab+€))b*+((b+a(aa)*(ab+€))b*b((a+a(aa)*(ab+€))b*b)*(€+(a+a(aa)*(ab+€))b*)



Convert below From FA to Regex





2.17 Arden's Theorem

"If P and Q are two RE over alphabet Σ, and if P does not contain null string ϵ, then the following equation in R is given by $R = Q + RP$ has a unique solution, i.e. $R = QP^*$."

It means, eqn in the form of $R = Q + RP$ can be replaced by $R = QP^*$.

Proof:

a) Proof of solution:

Let us take the equation, $R = Q + RP$ --(1)

Now, replacing R by $R = QP^*$,

$R = Q + QP^*P$

$= Q(\epsilon + P^*P)$

$\therefore R = QP^*$ proved $(\because \epsilon + R^*R = R^*)$

b) Proof for the only solution

Consider, $R = Q + RP$

$= Q + (Q+RP)P$ $(\because R = Q+RP)$

$= Q + QP + RP^2$

Again replace R by $Q + RP$

$R = Q + QP + (Q+RP)P^2$

$= Q + QP + QP^2 + RP^3$

$= Q + QP + QP^2 + \cdots QP^n + RP^{n+1}$

GURUKUL

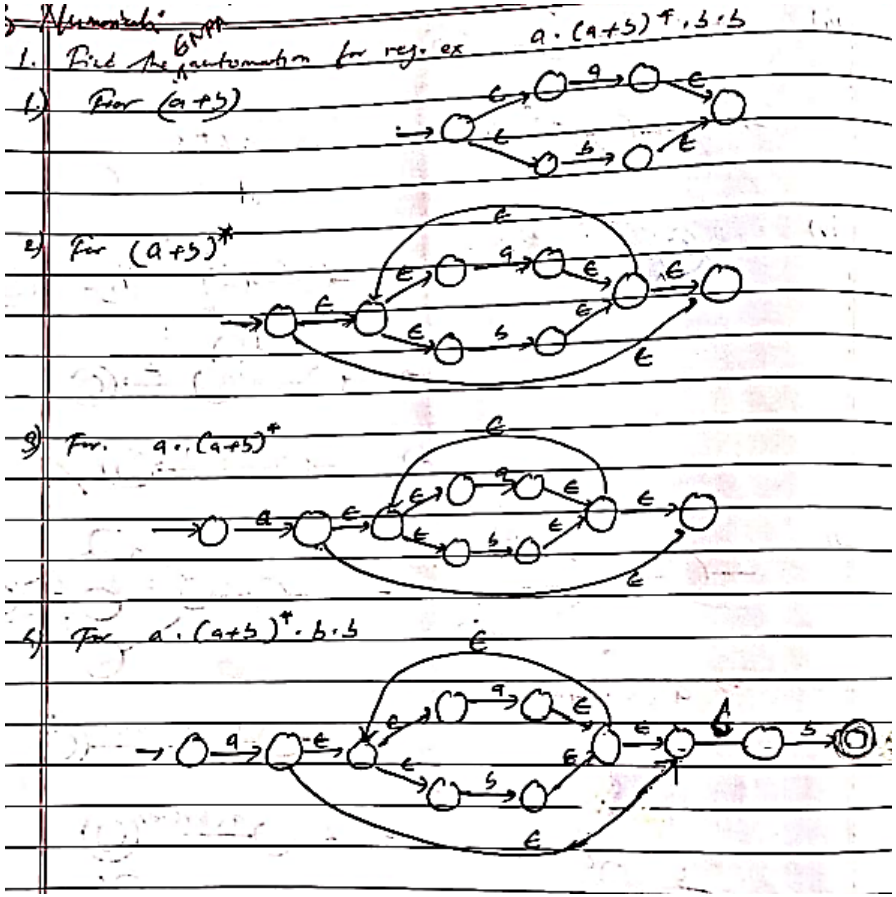
$= Q + QP + QP^2 + \cdots QP^n + QP^*P^{n+1}$ $(\because R + QP^*)$

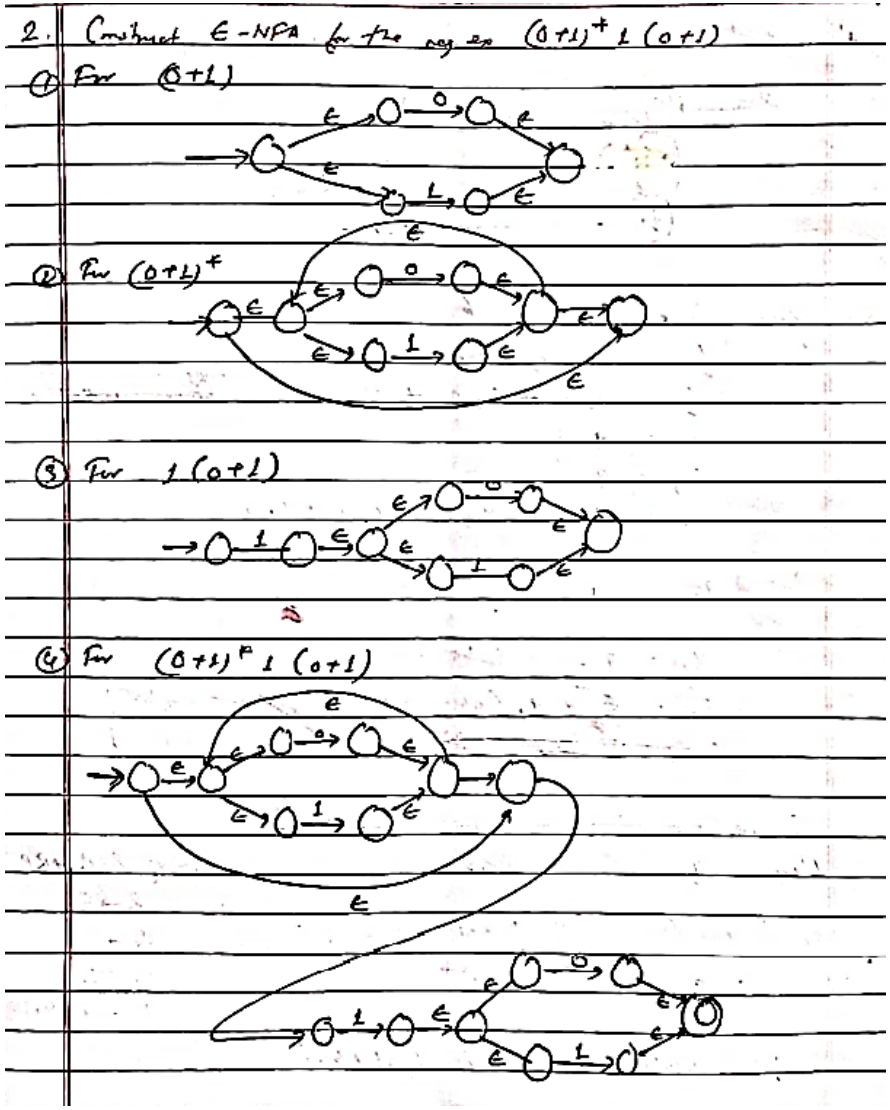
$= Q(\epsilon + P + P^2 + \cdots P^n + P^*P^{n+1})$

$= QP^*$ $(\because P^* \text{ is the represent the closure of } P)$

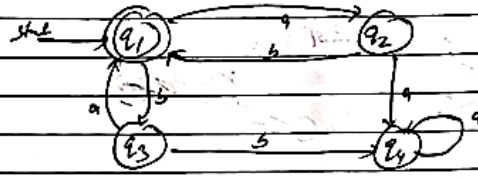
Hence proved

2.18 Numerical





3) Find the regular expression for transition diagram given in.



Sol": Using Arden's theorem,

Let us form the equations: (considering incoming edges)

$q_1 = q_2 b + q_3 a + \epsilon$

$q_2 = q_1 a$

$q_3 = q_1 b$

$q_4 = q_2 a + q_3 b + q_4 a + q_4 b$

Put q_2 and q_3 in q_1 eqn:

$q_1 = q_1 ab + q_1 ba + \epsilon$

$q_1 = \epsilon + q_1 (ab + ba)$

Here $q_1 = \epsilon + q_1 (ab + ba)$ is in the form of $R = Q + RP$

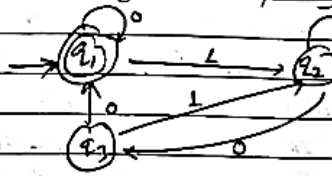
$R \to q_1$, $Q \to \epsilon$, $P \to (ab+ba)$

By Arden's theorem, $R = Q + RP \Rightarrow R = QP^*$

$\therefore q_1 = \epsilon (ab + ba)^*$

So, req^d regular expression is $(ab + ba)^*$

4) Construct a regex corresponding to the state diagram given as:



Sol": Let us form the equations:

$q_1 = q_1 0 + q_3 0 + \epsilon$

$q_2 = q_1 1 + q_2 1 + q_3 1$

$q_3 = q_2 0$

Now,

$q_2 = q_1 1 + q_2 1 + (q_2 0) 1$

$= q_1 1 + q_2 (1 + 01)$

$= q_1 1 (1 + 01)^*$

Again:

$q_1 = q_1 0 + q_3 0 + \epsilon$

$= q_1 0 + q_2 00 + \epsilon$

$= q_1 0 + (q_1 1 (1+01)^*) 00 + \epsilon$

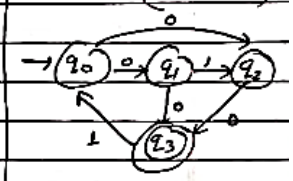
$= \epsilon + q_1 (0 + 1 (1+01)^* 00)$

$= \epsilon (0 + 1 (1+01)^* 00)^*$

$= (0 + 1 (1+01)^* 00)^*$

So regex is: $(0 + 1 (1+01)^* 00)^*$

Find regex for:-



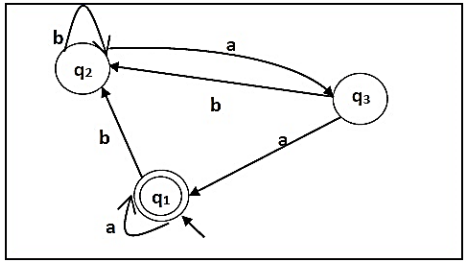
Here $R \to q_1$, $Q \to q_1 1$, $P = (1+01)$

Applying $R = QP^*$

$\therefore R = QP^*$ where $R \to q_1$, $Q \to \epsilon$, $P \to (0+1(1+01)^*00)$

Tutorial: Page 104, Ex: 4.16

Problem: Construct a regular expression to the automata given below –



Solution –

Here the initial state and final state is q_1.

The equations for the three states q1, q2, and q3 are as follows –

$q_1 = q_1a + q_3a + \varepsilon$ (ε move is because q1 is the initial state0

$q_2 = q_1b + q_2b + q_3b$

$q_3 = q_2a$

Now, we will solve these three equations –

$q_2 = q_1b + q_2b + q_3b$

$= q_1b + q_2b + (q_2a)b$ (Substituting value of q_3)

$= q_1b + q_2(b + ab)$

$= q_1b\ (b + ab)^*$ (Applying Arden's Theorem)

$q_1 = q_1a + q_3a + \varepsilon$

$= q_1a + q_2aa + \varepsilon$ (Substituting value of q_3)

$= q_1a + q_1b(b + ab^*)aa + \varepsilon$ (Substituting value of q_2)

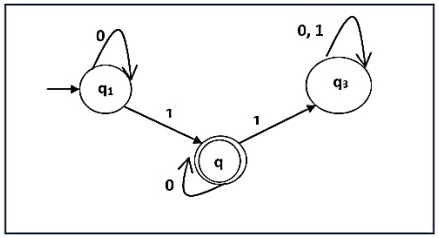
$= q_1(a + b(b + ab)^*aa) + \varepsilon$

$= \varepsilon\ (a+ b(b + ab)^*aa)^*$

$= (a + b(b + ab)^*aa)^*$

Hence, the regular expression is $(a + b(b + ab)^*aa)^*$

Problem: Construct a regular expression corresponding to the automata given below –



Solution –

Here the initial state is q_1 and the final state is q_2

Now we write down the equations –

$q_1 = q_10 + \varepsilon$

$q_2 = q_11 + q_20$

$q_3 = q_21 + q_30 + q_31$

Now, we will solve these three equations –

$q_1 = \varepsilon0^*$ [As, $\varepsilon R = R$]

So, $q_1 = 0^*$

$q_2 = 0^*1 + q_20$

So, $q_2 = 0^*1(0)^*$ [By Arden's theorem]

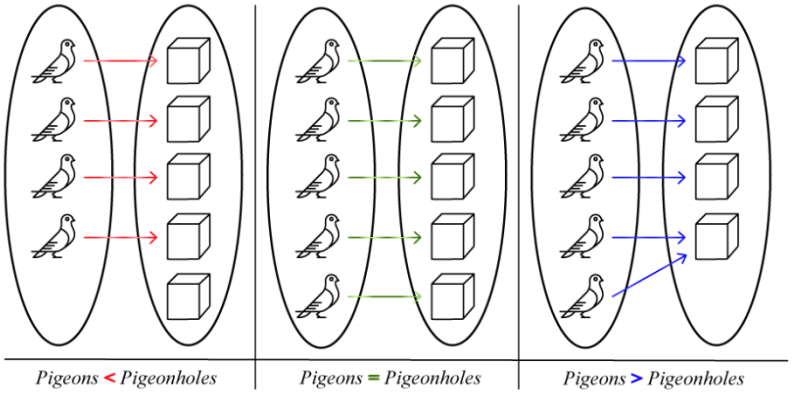
Hence, the regular expression is 0^*10^*.

Tutorial:

1. Construct an NFA for (ab/ba)*ab

2.19 The pigeonhole principle

The idea is this.
If there are n number of pigeonholes and (n+1) pigeons, and if all the pigeons come home to roost, then there will be a pigeonhole that contains more than one pigeon.



Pigeonhole Principle Diagram

The pigeonhole principle, also known as the Dirichlet principle, originated with German mathematician Peter Gustave Lejeune Dirichlet in the 1800s, who theorized that given m boxes or drawers and n > m objects, then at least one of the boxes must contain more than one object.
If m is the number of objects (pigeons) and n is the number of boxes (pigeonholes), and let f: m→ n be a function, then

The abstract formulation of the principle: Let X and Y be finite sets

- If m<n, then function is one-to-one but not onto.
- If m=n, then function is both one-to-one but onto (bijection).
- If m>n , then function is onto but not one-to-one

Example:
For instance, suppose there are 35 different time periods during which classes at the local college can be scheduled. If there are 679 different classes, what is the minimum number of rooms that will be needed?

Let $n = 679$ (pigeons) and $m = 35$ (pigeonholes)

Then $\left\lceil \frac{679}{35} \right\rceil = \lceil 19.4 \rceil = 20$

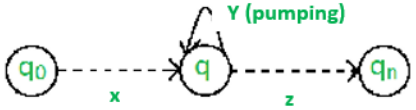
Therefore, the local college will need 20 different rooms to accommodate the different classes and periods.

2.20 Pumping Lemma for Regular Language

Statement

If L is a regular language, then L has a pumping length 'p' such that any string 's' where |s|≥ p may be divided into 3 parts s=xyz such that the following conditions must be true:

1. xyiz ∈L for every i ≥ 0
2. |y|> 0
3. |xy|≤ p



In simple terms, this means that if a string y is 'pumped', i.e., if y is inserted any number of times, the resultant string still remains in L.
Pumping Lemma is used as a proof for irregularity of a language. Thus, if a language is regular, it always satisfies pumping lemma. If there exists at least one string made from pumping which is not in L, then L is surely not regular.
The opposite of this may not always be true. That is, if Pumping Lemma holds, it does not mean that the language is regular.

Proof

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA recognizing a language L.
Let L is regular. Let n be the no of states.
Consider any string $\overset{s}{\omega}$ of length n or more, i.e. $\omega \geq n$.

Let $\overset{s}{\omega} = a_1 a_2 \ldots a_m$ be the string, and
$Q = \{q_0, q_1, \ldots q_m\}$ be the states.

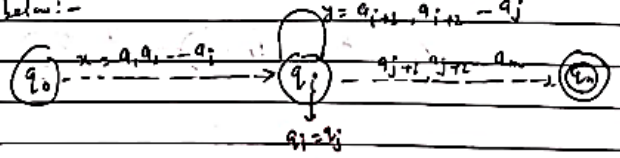
The transition function is: $\delta(q_0, a_1 a_2 \ldots a_i) = q_i$ for $i = 1, 2, \ldots m$

Q is the sequence of states in the path with the path value $\overset{s}{\omega} = a_1 a_2 \ldots a_m$.
As there are only n distinct states, d least two states in Q must coincide. (By pigeonhole principle).
Letus take q_i and q_j $(q_i = q_j)$. Then i and j satisfy the condition $0 \leq i < j \leq n$

Now let us break $\overset{s}{\omega} = xyz$ as follows: three substring:

$x = a_1 a_2 \ldots a_i$
$y = a_{i+1}, a_{i+2} \ldots a_j$
$z = a_{j+1}, a_{j+2} \ldots a_m$

The path with path value ω in the transition diagram of M is shown below:-



The automaton M starts from the initial state q_0.
On applying the string x, it reaches the state q_i.
On applying the string y, it comes back to q_i.
On applying z, it reaches to q_m, a final state.

Hence, $xy^i z \in L$ for $i \geq 0$ — condition 1

As every state in Q is obtained by applying an input symbol, $y \neq \epsilon$. Hence $|y| > 0$ — condition 2

As, $i \leq n$, $|xy| \leq p$ — condition 3
where p is the pumping length.

2.21 Steps for proving given language is not Regular using Pumping Lemma theory

To prove that a language is not regular using Pumping Lemma follow the below steps.

1. Assume that A is regular
2. It has to have a Pumping length (say p)
3. All strings longer than p can be pumped $|s| \geq p$.
4. Now find a string 's' in A such that $|s| \geq p$
5. Divide s into x,y,z
6. Show that $xy^i z \notin A$ for some i
7. Then consider all ways that s can be divided into xyz
8. Show that none of these can satisfy all the 3 pumping conditions at the same time.
9. s cannot be pumped == Contradiction.

2.22 Numerical

1. Show that $L = \{a^n b^n, n \geq 1\}$ is not regular.

Proof: Assume that L is regular, and the pumping length is P, and S is the string.

Here, $S = a^n b^n$

Let us consider $P = 7$

According to the Pumping Lemma statement, $|S| \geq P$, so we take here,

$S = a^P b^P = a^7 b^7$ "i.e. aaaaaaa bbbbbbb

This S can be divided in the form of xyz such that $xy^iz \in L$ for every $i \geq 0$, $|y| > 0$ and $|xy| \leq P$

Case 1: Let's divide S in xyz form where y is in the part of a.

a a a a a a a b b b b b b b

x | y | z

Taking $i=2$, $xy^iz = xy^2z$

$= aa\ aaaaaa\ a\ bbbbbbb$

$= a^{11} b^7$

Here, $xy^iz \notin A$ NA satisfied

$|y| > 0$ satisfied

$|xy| \leq P$ satisfied

Case 2: The y is in the part of b

a a a a a a a b b b b b b b

x | y | z

Taking $i=2$,

$xy^iz = xy^2z$

$= aaaaaaa\ bbbbbbb\ b$

$= a^7 b^{11}$

Here, $xy^iz \notin A$ NA satisfied.

$|y| > 0$ satisfied

$|xy| \leq P$ NA satisfied

Case 3: The y is in the part of both a and b

a a a a a a b b b b b b

x | y | z

Taking $i=2$,

$xy^iz = xy^2z$

$= aaaaa\ aabbaabb\ bbbbb$

$= a^7 b^2 a^2 b^7$

Here $xy^iz \notin A$ NA satisfied

$|y| > 0$ satisfied

$|xy| \leq P$ satisfied

From the above three cases, we can observe that none of the cases can satisfy all the three pumping conditions at the same time.

Hence, proved by contradiction, Given language is not regular.

2. Show that $A = \{yy \mid y \in \{0,1\}^*\}$ is not regular

Proof: Assume that A is regular, so must have pumping length.

Let pumping length = P and S is any string in the language. Let P = 7

Let us choose, $S = 0^P 1\, 0^P 1$

$= 0^7 1\, 0^7 1 = 00000001\,00000001$

According to Pumping lemma statement this S can be divided into the form of xyz such that $xy^iz \in A$ for every $i \geq 0$, $|y| > 0$ and $|xy| \leq P$.

case 1: Let's divide S in xyz where y is in the first half

$S = \underbrace{0}_{x}\underbrace{0000000}_{y}1\,\underbrace{00000001}_{z}$

Taking $i = 2$, $xy^iz = xy^2z$

$= 00\,0000000\,01\,00000001$

$\Rightarrow$ 1st half $\neq$ 2nd half

$\therefore xy^iz \notin A$ Not satisfied

$|y| > 0$ satisfied

$|xy| \leq P$ satisfied

case 2: Dividing S in xyz where y is in 2nd half

$S = \underbrace{00000001}_{x}\underbrace{0}_{y}\underbrace{0000001}_{z}$

~~Here $|xy| \leq P$ NA satisfied.~~

Taking $i = 2$, $xy^iz = xy^2z$

$= 000000001000000001$

NA satisfied $\Rightarrow$ 1st half $\neq$ 2nd half

$\therefore xy^iz \notin A$, $|y| > 0$ satisfied, $|xy| \leq P$ satisfied

Hence, proved by contradiction, given Language is not Regular.

3. Show that $L = \{a^n b^{2n} : n \geq 1\}$ is not regular

Proof: Assume that L is regular, and the pumping length is P, and S is the string.

Here, $S = a^n b^{2n}$

Let us consider $P = 7$.

~~According to the pumping lemma statement, $|xy| \leq P$~~

Let us choose, $S = a^P b^{2P} = a^7 b^{14}$

$= aaaaaaa\,bbbbbbbbbbbbbb$

Acc. to pumping lemma statement, S can be divided in xyz form, such that $xy^iz \in A$ and $|y| > 0$ and $|xy| \leq P$.

Case 1: Let us divide S into x, y, z such that $|xy| \leq P$ and $|y| > 0$ in pdf

$S = \underbrace{aaaaaaa}_{x}\underbrace{bbbbbbbbbbbbbb}_{z}$

Taking $i = 2$, $xy^iz = xy^2z$

$= aaaaaa\,aaaa\,bbbbbbbbbbbbbb$

$= a^9 b^{14}$

$\therefore xy^iz \notin A$ Not satisfied.

Case 2: Divide S into xyz, y is part of b such that $|xy| \leq P$ and $|y| > 0$

$S = \underbrace{aaaaaaa}_{x}\underbrace{bbbbbbbbbbbbbb}_{y\,z}$

Taking $i = 2$, $xy^iz = aaaaaaa\,bbbbbbbbbbbbbb$

$= a^7 b^{20}$

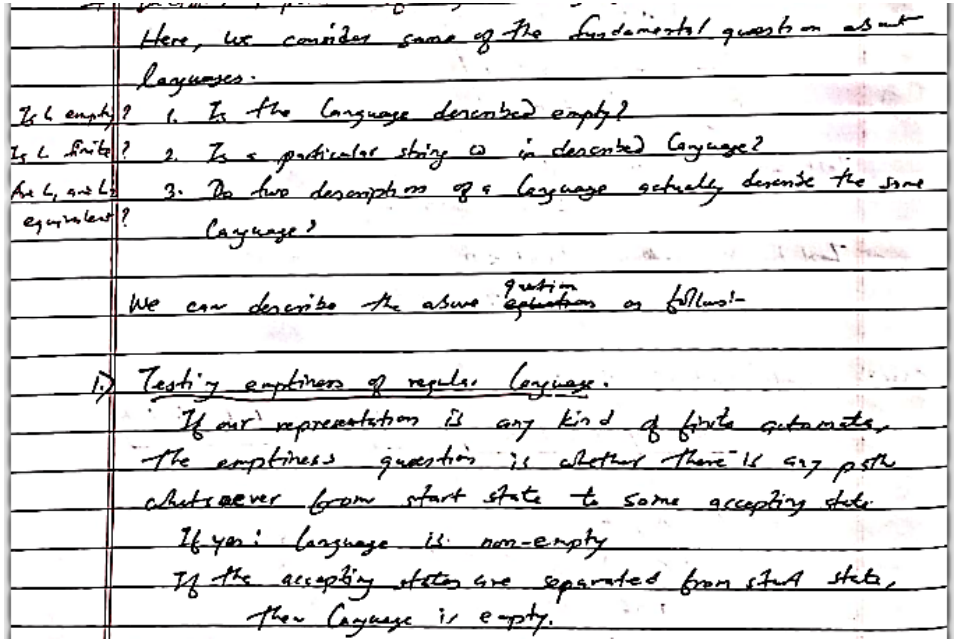
$\therefore xy^iz \in A$ | NA satisfied.

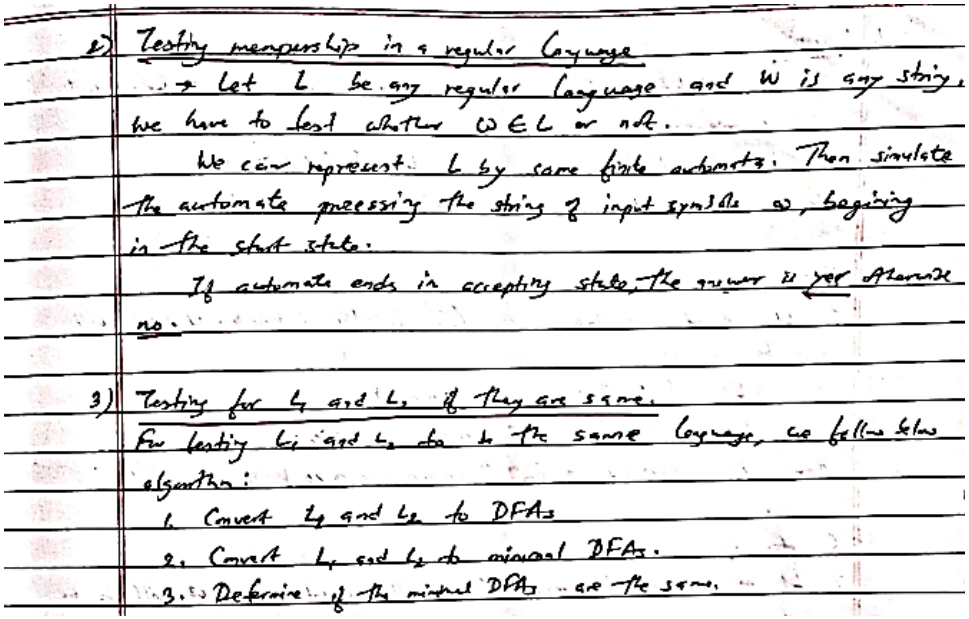
Hence proved by contradiction L is not regular.

Tutorial:
Prove that below languages are not regular.

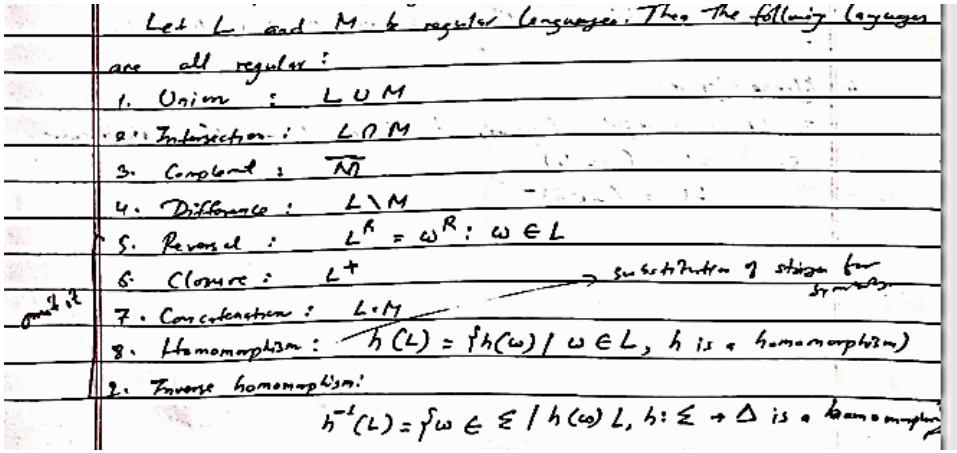
1. $L=\{a^n b a^n$ for $n=0,1,2,\ldots\ldots\}$
2. $L=\{a^n b a b^{n+1}$ for $n=1,2,3,\ldots\ldots\}$
3. $L=\{0^n 1^m 2^n$ for $n, m \geq n\}$
4. $L=\{0^n 1^m \mid n \leq m\}$
5. $L=\{0^n 1^{2n} \mid n \geq 1\}$
6. $L=\{0^n \mid n$ is a perfect square$\}$
7. $L=\{0^k \mid k$ is prime no$\}$

2.23 Decision properties of Regular Language





2.24 Closure Properties of Regular Language



1. Union:

If L1 and L2 are two regular Languages, their union L1 ∪ L2 will also be regular.

Eg. $L1 = \{a^n / n>0\}$ and $L2 = \{b^m / n>0\}$

Then $L3 = L1 \cup L2 = \{a^n \cup b^m / n>0\}$ is also regular

2. Intersection:

If L1 and L2 are two regular Languages, their intersection L1 ∩ L2 will also be regular.

Eg. $L1 = \{a^m b^n / n>0$ and $m>0\}$ and

$L2 = \{a^m b^n \cup b^n a^m / n>0$ and $n>0\}$

Then $L3 = L1 \cap L2 = \{a^m b^n / n>0$ and $n>0\}$ is also regular

3. Concatenation:

If L1 and L2 are two regular Languages, their concatenation L1.L2 will also be regular

Eg. $L1 = \{a^m / m>0\}$ and $L2 = \{b^n / n>0\}$

Then $L3 = L1.L2 = \{a^m . b^n / m>0$ and $n>0\}$ is also regular

4. Kleene Closure:

If L1 is a regular Language, its kleene closure $L1^*$ will also be regular

Eg. $L1 = (a \cup b)$

$L1 = (a \cup b)^*$

2.25 Numerical

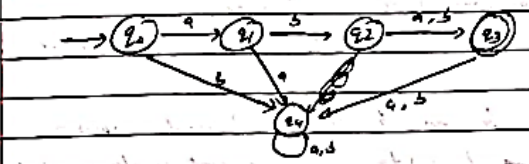
Show that If L is regular then complement of $L(\text{or } \bar{L})$ is also regular.

→ If L is a regular Language, then $\bar{L}$ is also regular.

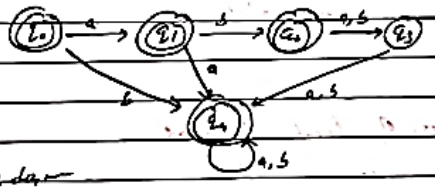
Let us consider a DFA that accepts only the strings aba and abb.

The state diagram is:-

$L = \{w \in \{a,b\}^+ :$ w is aba or abb$\}$



To find out the req^d FA which accepts strings other than aba and abb, we convert every non-final state to final state, and every final state to non-final.



$\bar{L} = \{w \in \{a,b\}^+ :$ w is not aba or abb$\}$

This is the complement of above state diagram, which accepts all other strings except aba and abb.

Thus we can say that if L is regular, then $\bar{L}$ is also regular.

End of Chapter 2